AI-POWERED VALET TRANSPARENCY AND VEHICLE SECURITY PLATFORM

A Phase 2 (60%) Progress Report submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

in

Computer Science and Engineering
Specialization in Artificial Intelligence and Machine Learning

By

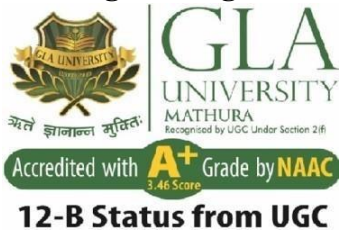
Mahesh (2415500275)
Jagesh Goyal(2415500209)

Group No. 54

Under the Guidance of

Dr. Gourav Bathla
Associate Professor and Associate Head, CSE (AI/ML)

Department of Computer Science and Engineering
Institute of Engineering & Technology



GLA University
Mathura – 281406, Uttar Pradesh, INDIA

October, 2026

DECLARATION

We hereby declare that the work which is being presented in the B.Tech. Project titled **"AIPowered Valet Transparency and Vehicle Security Platform",** in partial fulfilment of the requirements for the award of the Bachelor of Technology in Computer Science and Engineering with Specialization in Artificial Intelligence and Machine Learning, and submitted to the Department of Computer Science and Engineering of GLA University, Mathura, is an authentic record of our own work carried out under the supervision of **Dr. Gourav Bathla,** Associate Professor and Associate Head, CSE (AI/ML), GLA University, Mathura.

The contents of this project report, in full or in part, have not been submitted to any other Institute or University for the award of any degree. All external sources of information, libraries and documentation used during this work have been acknowledged in the References section.

This document is a second progress report describing approximately sixty per cent of the planned project work. Where a feature is partially implemented, simulated, or still under development, this report states so explicitly rather than presenting it as complete.

Candidate

Sign: _____

Mahesh Sorout
B.Tech CSE (AIML)
Univ. Roll No.:2415500275

Candidate

Sign: _____

Jagesh Goyal
B.Tech CSE (AIML)
Univ. Roll No.: 2415500209

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who supported and guided us through the first two phases of this project. First and foremost, we extend our heartfelt thanks to our respected mentor, Dr. Gourav Bathla (Associate Professor and Associate Head, CSE AI/ML), for continuous encouragement, valuable feedback and insistence on technical honesty at every review. His guidance shaped both the engineering decisions and the way this report distinguishes verified functionality from work is still in progress.

We are grateful to the Department of Computer Science and Engineering, GLA University, Mathura, for providing the laboratory facilities, software resources and academic environment required to build and test a full-stack application of this scope. We also thank the project coordinators of the B.Tech. CSE (AI/ML) programme for their support and for the timely scheduling of progress reviews.

We acknowledge the open-source community behind React, Flask, PostgreSQL, OpenCV, scikit-learn and Leaflet. This project stands on their documentation, issue trackers and reference implementations, all of which are cited in the References section.

Finally, we thank our families and classmates for their patience and encouragement during the long testing and debugging cycles that this milestone required.

ABSTRACT

Valet parking is a trust-intensive service. When a customer surrenders a vehicle at a hotel, hospital or event, that vehicle passes through a custody window — from handover at the drop-off point to return at pickup — during which the owner is physically absent and usually has no information beyond a paper token. If the vehicle is moved without authorisation, damaged, or returned late, the customer and the operator frequently disagree about what happened, because neither side holds a verifiable record of the interval. Existing valet operations address this with manual logs and verbal assurance, which are easy to lose and impossible to audit.

This project builds an AI-powered valet transparency and vehicle security platform that makes that custody window observable. The system is implemented as a separated React– Flask–PostgreSQL application. A customer registers a vehicle and receives a digital valet ticket with a server-generated QR code; a valet operator checks the vehicle in, assigns it to a geofenced parking zone, and records condition photographs; the platform then captures positioning telemetry, classifies each fix against the real zone polygon, raises security alerts when the vehicle leaves its permitted area or behaves abnormally, compares pickup photographs against the drop-off baseline using an OpenCV image-differencing pipeline, predicts retrieval waiting time, and preserves every action in an append-only audit trail. Customers, valet operators and managers each receive a dashboard scoped to their role.

At the sixty per cent milestone described in this report, the platform is functionally complete across eight implemented modules: authentication and role-based access control, vehicle and ticket management, custody sessions with a validated state machine, condition assessment, GPS tracking with a Leaflet map and geofence classification, centralised security alerts, retrieval-time estimation, and an audit trail. The system is verified by 1,088 backend tests and 671 frontend tests, all passing, together with a 24-step end-to-end workflow harness. Three points are stated explicitly because they bound what the results mean: the computer-vision component uses classical OpenCV differencing and is not a trained damage classifier; all positioning telemetry in this milestone is simulated and labelled as such on every stored row

and in the user interface; and the machine-learning components are trained on synthetic data and reported against baselines rather than presented as production accuracy.

The report documents the software requirement analysis, system design, implementation, interface, testing evidence, current progress and the remaining forty per cent of work, including hardware telemetry integration, mobile capture, and field validation of the prediction components.

TABLE OF CONTENTS

Declaration....ii
Certificate....iii
Acknowledgement....iv
Abstract....v
Table of Contents....vii
List of Tables....ix
List of Figures....xi

CHAPTER 1: INTRODUCTION....1
1.1 Background and Overview....1
1.2 Problem Statement and Motivation....2
1.3 Project Objectives....2
1.4 Scope of the Project....4
1.5 Existing System and Its Limitations....5
1.6 Proposed System....6
1.7 Comparison with Existing Approaches....7
1.8 Organization of the Report....7
CHAPTER 2: SOFTWARE REQUIREMENT ANALYSIS....9
2.1 Introduction to Requirement Analysis....9
2.2 Stakeholders and User Roles....9
2.3 Functional Requirements....10
2.4 Non-Functional Requirements....13
2.5 Hardware and Software Requirements....14
2.6 Feasibility Study....15
2.7 Use Case Descriptions....16
2.8 Module Descriptions....18
CHAPTER 3: SOFTWARE DESIGN....19
3.1 System Architecture Overview....19
3.2 Frontend–Backend Communication....20
3.3 API Architecture and Request Flow....21
3.4 Database Design and Entity Relationships....23
3.5 Data Flow Diagrams....25
3.6 Use Case Diagram....26
3.7 Sequence Diagram....27
3.8 Security and Authentication Design....28
3.9 GPS, Geofencing and Custody-Session Workflow....30

CHAPTER 4: IMPLEMENTATION AND USER INTERFACE.....33
4.1 Technology Stack.....33
4.2 Backend Implementation.....34
4.3 Frontend Implementation.....35
4.4 Authentication and Role-Based Access.....36
4.5 Vehicle Registration and Valet Ticket Workflow.....37
4.6 Vehicle Custody and Inspection Workflow.....39
4.7 Computer Vision and Damage-Detection Module.....41
4.8 GPS Tracking, Map and Geofencing Module.....43
4.9 Retrieval-Time Prediction Module.....45
4.10 Database Integration.....45
4.11 Current Implementation Status.....46
CHAPTER 5: TESTING AND VALIDATION.....51
5.1 Testing Strategy.....51
5.2 Backend API Testing.....51
5.3 Database and Integration Testing.....52
5.4 Frontend Workflow Testing.....53
5.5 Authentication and Authorization Testing.....54
5.6 Computer Vision Test Cases.....55
5.7 GPS Simulation and Geofence Test Cases.....56
5.8 Retrieval-Time Prediction Validation.....57
5.9 Test Results and Known Issues.....58
CHAPTER 6: CURRENT PROGRESS, CHALLENGES AND REMAINING WORK.....59
6.1 Overall Project Progress.....59
6.2 Completed Work.....60
6.3 Partially Completed Work.....60
6.4 Technical Challenges and Resolutions.....60
6.5 Remaining 40% Work Plan.....61
6.6 Timeline for Completion.....62
CHAPTER 7: CONCLUSION AND FUTURE SCOPE.....64
7.1 Current Achievements.....64
7.2 Limitations.....64
7.3 Future Enhancements.....65
7.4 Final Conclusion.....65

REFERENCES.....67

APPENDIX A: API Endpoint Summary.....69
APPENDIX B: Database Schema Summary.....71
APPENDIX C: Test Evidence.....72
APPENDIX D: Additional Screenshots.....74

LIST OF TABLES

Table 1.1: Project objectives and their status at the 60% milestone....3
Table 1.2: Limitations of the existing manual and ticketing-only systems....5
Table 1.3: Components of the proposed system and their responsibility....6
Table 1.4: Capability comparison with alternative approaches....7
Table 2.1: Stakeholders, roles and their responsibilities....10
Table 2.2: Functional requirements and implementation status....10
Table 2.3: Non-functional requirements and how they are addressed....13
Table 2.4: Software requirements....14
Table 2.5: Hardware requirements....14
Table 2.6: Use case UC-1 — check in a vehicle....16
Table 2.7: Use case UC-2 — view live tracking....17
Table 2.8: Use case UC-3 — verify a damage finding....17
Table 2.9: Module descriptions and implementation status....18
Table 3.1: Layers and their allowed dependencies....20
Table 3.2: API conventions applied uniformly....21
Table 3.3: Endpoint groups and representative operations....22
Table 3.4: Core entities and their role....24
Table 3.5: Role capability matrix (enforced by the backend)....29
Table 3.6: Custody states, meaning and projection....30
Table 3.7: Distance rule and its configured thresholds....31
Table 3.8: Simulation scenarios and their intended verdicts....32
Table 4.1: Implemented technology stack....33
Table 4.2: Backend package structure....34
Table 4.3: Implemented routes and audience....35
Table 4.4: Authentication module summary....36
Table 4.5: Ticket module summary....38
Table 4.6: Inspection module summary....39
Table 4.7: Vision pipeline stages....42
Table 4.8: Tracking module summary....43
Table 4.9: Retrieval module summary....45
Table 4.10: Migration sequence....45
Table 4.11: Module status and supporting evidence....46
Table 5.1: Test layers and their purpose....51
Table 5.2: Backend test modules and results....52
Table 5.3: Database invariants under test....53
Table 5.4: Frontend test groups....53
Table 5.5: Authentication and authorisation test cases....54
Table 5.6: Computer-vision test cases....55
Table 5.7: Geofence rule test cases....56
Table 5.8: Scenario-level verdict distribution (fresh session per scenario)....56
Table 5.9: Retrieval model evaluation (held-out, synthetic data)....57
Table 5.10: Consolidated test results....58
Table 5.11: Known issues and their status....58

Table 6.1: Weighted progress against the project plan.....59
Table 6.2: Partially completed work and what remains.....60
Table 6.3: Technical challenges encountered and their resolution.....61
Table 6.4: Planned work for the remaining phase.....61
Table 6.5: Proposed schedule for the final phase.....62
Table 7.1: Future enhancements and their purpose.....65

LIST OF FIGURES

Figure 1.1: The custody record chain from drop-off to pickup.....6
Figure 3.1: System architecture.....19
Figure 3.2: Entity–relationship diagram of the core schema.....23
Figure 3.3: Level-0 data flow diagram (context).....25
Figure 3.4: Level-1 data flow diagram.....26
Figure 3.5: Use case diagram for the four roles.....27
Figure 3.6: Sequence diagram for vehicle check-in.....28
Figure 3.7: Custody-session state machine.....30
Figure 4.1: Sign-in screen with the session-rejection notice used after an expiry.....37
Figure 4.2: Customer vehicle list with registration entry point.....38
Figure 4.3: Digital valet ticket with server-rendered QR and custody summary.....39
Figure 4.4: Vehicle check-in — zone selection and odometer capture.....40
Figure 4.5: Inspection capture screen for drop-off photographs.....41
Figure 4.6: Condition report with before/after evidence and verification controls.....41
Figure 4.7: Live tracking — geofence polygon, route, position markers and legend.....44
Figure 4.8: Customer dashboard — active custody, vehicle and retrieval state.....47
Figure 4.9: Valet operator dashboard — active vehicles, pending pickups and queue.....47
Figure 4.10: Manager dashboard — filtered metrics, alerts and export.....48
Figure 4.11: Security alerts with severity, source and triage controls.....48
Figure 4.12: Audit trail with actor, action and entity details.....49
Figure 4.13: System status — live dependency health and version information.....49

CHAPTER 1 INTRODUCTION

1.1 Background and Overview

Valet parking is a service in which a customer hands over a vehicle to an operator who parks it on the customer's behalf and returns it later. It is common at hotels, hospitals, shopping centres, wedding venues and corporate events, and it is valuable precisely because it removes a chore. That convenience depends on trust: between the moment the key is handed over and the moment the vehicle is returned, the owner is physically separated from the asset and from any observation of how it is treated.

This interval is referred to throughout this report as the **custody window**. It begins at dropoff, when a valet accepts the vehicle, and ends at pickup, when the vehicle is returned. In a conventional operation, the only artefacts produced during this window are a paper token and a handwritten register entry. Neither records where the vehicle was kept, whether it moved, who moved it, what condition it was in at handover, or how long each stage took. If something goes wrong — a scratch discovered at pickup, a delayed return, a vehicle photographed in an unexpected location — both parties are reduced to assertion against assertion.

Modern smartphones, inexpensive GPS receivers, ubiquitous cameras and open mapping data make it technically feasible to observe that window end to end. What is missing in practice is not sensing hardware but a system that turns observations into a coherent, auditable record, and presents it to the three parties who need it: the owner, the operator and the manager. This project builds that system.

The platform is implemented as a separated web application. A React single-page application provides role-specific dashboards; a Flask REST API enforces business rules and authorisation; PostgreSQL stores the custody record, telemetry and inspection evidence; OpenCV performs photographic comparison; scikit-learn provides two bounded analytical components; and Leaflet renders the parking zone, the route and the current position over OpenStreetMap imagery.

1.2 Problem Statement and Motivation

The problem this project addresses can be stated in one sentence: **during the custody window of a valet-parked vehicle, neither the owner nor the operator possesses a verifiable, time-stamped, tamper-resistant account of what happened to the vehicle.**

That single gap produces four recurring and concrete failures.

1. **Disputed condition.** A scratch, dent or kerb mark noticed at pickup is attributed by the customer to the parking period and denied by the operator, because the drop-off condition was never recorded objectively. Without a baseline photograph and a comparison, the argument cannot be resolved by evidence.
2. **Unobserved movement.** A vehicle may be moved to a second location, used for an errand, or driven at speed inside a cramped yard. The customer has no means of knowing, and the operator has no incentive to record it.
3. **Opaque waiting.** A customer waiting for a vehicle has no information about where the retrieval stands. The quoted "ten minutes" is an estimate offered verbally and is not derived from any measured queue.
4. **No accountability trail.** Even where a manager suspects misconduct, there is no audit log correlating a person, a time, a vehicle and an action. Investigations rely on memory and closed-circuit footage that may not exist.

The motivation for the project is therefore not the addition of another parking application. It is the construction of a **transparency layer** over an existing service: a system whose primary output is trustworthy evidence, and whose secondary output is the operational convenience (tickets, dashboards, estimates) that makes operators willing to adopt it.

Why transparency, and not merely tracking
A tracking screen that shows a dot on a map is a monitoring tool for the operator. It does not resolve a dispute, because the operator controls the data and the customer cannot verify it.
The design decision taken here is that every observation must be (i) stored with its provenance, (ii) classified by a stated, inspectable rule set rather than an opaque score, and (iii) readable by the customer through their own account. That combination — provenance, explainability, access — is what converts monitoring into transparency.

1.3 Project Objectives

The objectives below were agreed at the start of the project. They are listed with their status at the sixty per cent milestone so that this report does not blur delivered work with planned work. Table 1.1 sets out each objective beside the artefact that demonstrates it and its status at this milestone, so that delivered work and planned work can be told apart.

Table 1.1: Project objectives and their status at the 60% milestone

ID	Objective	Deliverable evidence	Status
O1	Digitise the valet transaction: issue a verifiable digital ticket with a server-generated QR code in place of a paper token.	Ticket service, QR generation, */api/v1/tickets*, ticket detail screen.	Completed
O2	Open and manage a custody session with an enforced lifecycle, so an invalid state change cannot be written.	Custody state machine with an allow-list, *PATCH /custody-sessions*, 409 on illegal moves.	Completed
O3	Record the vehicle's condition at drop-off and compare it at pickup using computer vision, with human verification of findings.	OpenCV comparison pipeline, inspection endpoints, condition-report screen, manager-only verification.	Completed
O4	Observe position during custody and classify it against the real parkingzone polygon, raising alerts on anomalies.	GPS ingestion, geofence rule engine, Leaflet tracking map, alert feed.	Completed
O5	Give each stakeholder a role-scoped dashboard; customer, valet operator and manager see only what their role permits.	Three dashboards, JWT + RBAC, per-endpoint role guards, 401/403 tests.	Completed
O6	Preserve an append-only audit trail so every action is attributable to a person and a time.	Audit service, database trigger blocking UPDATE/DELETE, audit screen, timeline endpoint.	Completed
O7	Estimate retrieval waiting time from queue state and show the basis of the estimate.	Retrieval model, */retrieval/request*, estimate with confidence and explanation.	Completed (synthetic data)
O8	Integrate real GPS hardware and on-device camera capture for a production deployment.	Telemetry-source abstraction exists; hardware integration not built.	Planned — 40%
O9	Validate the analytical components against real yard data.	Evaluation harness exists; no real-world dataset is available to the team.	Planned — 40%

Objectives O1 to O7 constitute the functional core and are demonstrated in Chapter 4 and tested in Chapter 5. Objectives O8 and O9 are explicitly outside this milestone; Section 1.4 explains why, and Section 6.5 plans them.

1.4 Scope of the Project

1.4.1 What the system does in this milestone

- **Account and access management** — customer self-registration, administratorprovisioned staff accounts, JWT authentication with refresh, sign-out revocation, four roles.

- **Vehicle registry** — owners register vehicles with a plate number, make, model and year; vehicles are listed, edited and retired.
- **Digital valet ticket** — issued by a valet operator for a registered vehicle, carrying a unique ticket code, a QR payload and a server-rendered PNG.
- **Custody lifecycle** — check-in assigns a parking zone and opens a session; the session then advances through an explicit state machine to completion.
- **Condition assessment** — multi-angle drop-off photographs, pickup photographs, perangle comparison, candidate findings labelled "potential damage", and manager verification.
- **Positioning and geofencing** — telemetry ingestion, per-fix classification against the zone polygon with a stated tolerance, track history, and a live map.
- **Alerts** — geofence and movement anomalies raised as alerts, triaged by managers, with per-recipient read state.
- **Retrieval estimation** — a queue-aware estimate returned when a customer requests the vehicle, with its contributing factors.
- **Dashboards and reporting** — customer, operator and manager views; aggregate metrics; CSV and JSON export.
- **Audit** — append-only record of actions with actor, entity, before/after values and request context.

1.4.2 What is explicitly out of scope for this milestone

- **Real GPS hardware.** No tracker is attached to a vehicle. All telemetry in this milestone is generated by a documented simulator, stored with *source = SIMULATED*, and labelled in the interface. The ingestion path is the same code a hardware feed would use.
- **Mobile applications.** The interface is a responsive web application; native camera capture is not implemented. Photographs are uploaded through the browser.
- **Payments and billing.** The platform records custody; it does not charge for it.
- **Production deployment, multi-tenancy and horizontal scale.** The system runs locally on a developer machine for demonstration and evaluation.
- **Real-world model validation.** The security classifier and the retrieval estimator are trained on synthetic data generated inside this repository. They are evaluated against baselines and are not presented as production accuracy.

This boundary is deliberate. A student project that claims hardware integration it cannot demonstrate is less useful to an evaluator than one that states its boundary precisely and implements everything inside it to a testable standard.

1.5 Existing System and Its Limitations

The existing system, in the operations this project targets, is a manual valet desk. Its workflow is well understood and varies little: the customer receives a paper or plastic token bearing a number; an operator records the plate and a bay number in a register; the key is stored in a key box; the vehicle is retrieved when the token is presented. Some sites add spreadsheets, and a few add commercial valet-management software that handles ticketing and dispatch.

Table 1.2 lists what the manual desk and a ticketing-only product each provide, and the consequence of the gap for the owner, the operator and the manager.

Table 1.2: Limitations of the existing manual and ticketing-only systems

Aspect	Manual desk	Ticketing-only software	Consequence
Condition record	None, or a verbal note	Free-text note at most	Damage disputes cannot be settled by evidence
Position knowledge	Bay number written by hand	Bay number stored	No evidence the vehicle stayed in its bay
Movement record	None	None	Unauthorised movement is undetectable
Time record	Often absent	Check-in and checkout times	Duration disputes and no per-stage analysis
Accountability	Register signature	Operator account	Actions not individually attributable or tamperevident
Customer visibility	None while waiting	Token status at best	Customer cannot see progress or condition
Auditability	Paper, easily lost	Editable database rows	Records can be altered after the fact

Two limitations deserve emphasis because they motivate specific design decisions in this project. First, **paper and editable databases are not evidence**: a record that can be altered, or lost, cannot settle a dispute. The audit trail in this system is therefore append-only and enforced by a database trigger, not by convention. Second, **a bay number is a claim, not an observation**: it records where somebody said the vehicle would be, not where it was. The geofence in this system evaluates measured coordinates against the actual polygon of the zone.

1.6 Proposed System

The proposed system replaces the token-and-register workflow with a chain of verifiable records. Figure 1.1 shows the conceptual chain; each link is implemented by a module described in Chapter 4.

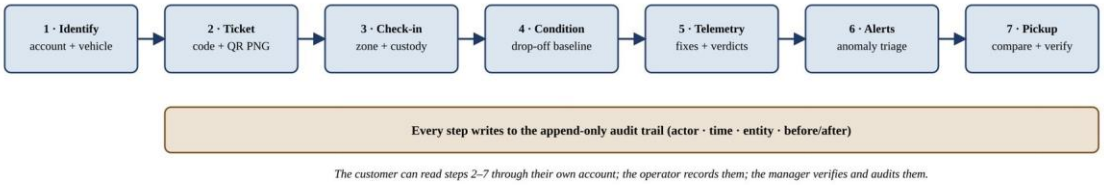


Figure 1.1: The custody record chain from drop-off to pickup

The chain is designed so that no single actor can silently rewrite history. The operator records the condition, but cannot verify their own damage findings; the manager verifies, but cannot alter the photographs; the customer sees the same records the operator sees, through their own account; and the audit log records all three, immutably.

1.6.1 Component summary

Table 1.3 summarises the components of the proposed system and assigns each a single responsibility in the chain.

Table 1.3: Components of the proposed system and their responsibility

Component	Technology	Responsibility
Web client	React 19, Vite 8, Leaflet	Role-specific dashboards, ticket and QR display, live map, inspection capture and reports.
Application programming interface	Flask (Python 3.13)	Authentication, authorisation, validation, business rules, response envelope, 63 versioned endpoints.
Domain services	Python modules	Ticket and custody lifecycle, inspections, geofence classification, alerts, retrieval, dashboards, audit.
Relational store	PostgreSQL 17	15 domain entities, constraints, append-only audit enforcement, eight Alembic migrations.
Vision pipeline	OpenCV, NumPy	Alignment, illumination normalisation, differencing, region segmentation, finding extraction.
Analytical models	scikit-learn	Prototype security classification of fixes; retrieval-time estimation with baseline comparison.
Telemetry simulator	Python	Six documented movement scenarios producing labelled SIMULATED fixes through the real ingestion path.
Media store	Local filesystem behind */media*	Uploaded inspection photographs and server-rendered QR PNGs, served with validation.

1.7 Comparison with Existing Approaches

Table 1.4 compares the platform with the three realistic alternatives a facility manager would consider. The comparison is restricted to capabilities that this repository actually implements or explicitly plans; no claim is made about commercial products' internal features beyond their published descriptions.

Table 1.4: Capability comparison with alternative approaches

Capability	Paper token	Ticketing software	Commercial valet apps	This platform
Digital ticket with QR	No	Yes	Yes	Yes
Custody state machine enforced	No	Partial	Vendor-specific	Yes
Condition baseline + comparison	No	No	Rarely	Yes (OpenCV)
Geofenced position monitoring	No	No	Add-on	Yes (simulated feed)
Explainable anomaly verdicts	No	No	Opaque scoring	Yes (rule ladder)
Append-only audit	No	No	Vendor-dependent	Yes (DB trigger)
Customer-visible evidence	No	Status only	Partial	Yes
Cost to a small facility	Low	Licence	Subscription	Open source
Validated on real yard data	—	—	Vendor claims	**Not yet — planned**

The final row is included deliberately. The one place where a commercial product legitimately surpasses this work is field validation, and the report does not pretend otherwise.

1.8 Organization of the Report

- **Chapter 1 — Introduction** states the problem, objectives, scope and the comparison with existing practice.
- **Chapter 2 — Software Requirement Analysis** records stakeholders, functional and non-functional requirements, hardware and software requirements, feasibility, use cases and module descriptions.
- **Chapter 3 — Software Design** presents the architecture, communication and API conventions, database design with the entity–relationship model, data-flow diagrams, the use-case and sequence diagrams, the security design and the geofence workflow.

- **Chapter 4 — Implementation and User Interface** documents each implemented module with its purpose, files, endpoints, entities, inputs and outputs, screenshots and known limitations.
- **Chapter 5 — Testing and Validation** reports the testing strategy and the executed results, with test-case tables and the measured output of the analytical components.
- **Chapter 6 — Current Progress, Challenges and Remaining Work** justifies the sixty per cent figure, records the technical problems actually encountered and their resolutions, and plans the remaining work.
- **Chapter 7 — Conclusion and Future Scope** summarises achievements, states limitations and sets out future enhancements.
- **References** lists the official documentation, standards and repositories used. **Appendices** provide the API summary, database schema summary, test evidence, abbreviations and additional screenshots.

CHAPTER 2 SOFTWARE REQUIREMENT ANALYSIS

2.1 Introduction to Requirement Analysis

Requirement analysis for this project began from the custody problem described in Chapter 1 rather than from a feature list. Each requirement was traced to a failure mode it prevents, and each was checked against what the platform can actually verify. This produced a smaller requirement set than a generic parking application would have, but every requirement in it is testable and every implemented requirement has a corresponding test in Chapter 5.

Requirements were gathered from three sources: (i) an analysis of the manual valet workflow as practised at commercial sites, (ii) the constraints imposed by the academic brief — separated React and Flask tiers, PostgreSQL, OpenCV and a mapping library — and (iii) an explicit engineering policy of refusing requirements whose evidence cannot be produced without hardware the team does not have. The third source removed several attractive but unprovable requirements, such as real-time engine telemetry and biometric operator identification.

The result is organised as functional requirements (Section 2.3), non-functional requirements (Section 2.4), platform requirements (Section 2.5), a feasibility study (Section 2.6), detailed use cases (Section 2.7) and module descriptions (Section 2.8). Each functional requirement carries an implementation status so that the requirement baseline and the current build can be compared directly.

2.2 Stakeholders and User Roles

Four authenticated roles are implemented. A fifth, unauthenticated visitor, can reach only the sign-in, registration and system-status screens. Role determines both the navigation the interface renders and — decisively — the authorisation checks the backend enforces on every request.

Table 2.1 defines each role — the four authenticated roles and the unauthenticated visitor — with its responsibilities and the screens it typically uses.

Table 2.1: Stakeholders, roles and their responsibilities

Role	Who	Primary responsibilities	Typical screens

Customer	Vehicle owner or authorised driver	Register and manage own vehicles; view own tickets and QR; track own vehicle; read security alerts about it; view condition reports; request retrieval; see the estimate.	Customer dashboard, My Vehicles, Tickets, Live Tracking, Alerts, Condition report
Valet Operator	Yard staff who take and return vehicles	Issue tickets; check vehicles in and assign a zone; capture drop-off and pickup photographs; move custody forward; process the retrieval queue; complete handover.	Valet dashboard, Vehicle Check-in, Inspection Capture, Retrieval queue
Valet Manager	Yard supervisor	Everything an operator does, plus: verify or reject damage findings, triage and resolve alerts, edit zones, view analytics, read the audit trail, export reports.	Manager dashboard, Alerts triage, Reports, Audit Log
Administrator	Platform owner	Create staff accounts; full oversight; all manager capabilities; system health and configuration.	Manager screens, System Status, user administration
Visitor	Unauthenticated user	Sign in, register a customer account, view system status.	Login, Register, System Status

The separation between operator and manager is not cosmetic. The brief and the security design require that the person who photographs a vehicle cannot also confirm that the damage they found is real; verification is restricted to manager and administrator roles, and that restriction is enforced server-side (Section 5.5).

2.3 Functional Requirements

Table 2.2 lists the functional requirements with their status at this milestone. ”Completed” means implemented and covered by automated tests; “Partial” means the capability exists but a documented sub-part is missing; “Planned” means not built.

Table 2.2: Functional requirements and implementation status

ID	Requirement	Priority	Status
FR-01	A visitor can register a customer account with email, password, name and optional phone; passwords are never stored in plain text.	High	Completed
FR-02	Public registration can never create a staff role; attempted escalation is refused.	High	Completed

ID	Requirement	Priority	Status
FR-03	A user can sign in and receive an access token and a refresh token; an expired access token can be refreshed without reentering credentials.	High	Completed
FR-04	Sign-out revokes the session's tokens serverside, not merely in the browser.	High	Completed
FR-05	A customer can register, list, edit and retire their own vehicles; one customer cannot see another's vehicles.	High	Completed
FR-06	A valet can issue one digital ticket per vehicle, with a unique ticket code, QR payload and serverrendered PNG.	High	Completed
FR-07	Duplicate ticket issuance for a vehicle already holding an unstarted ticket is refused with a conflict.	Medium	Completed
FR-08	A valet can check a vehicle in, assigning a parking zone; a custody session opens and zone occupancy increments atomically.	High	Completed
FR-09	Custody status transitions are validated against an explicit allow-list; illegal transitions are refused.	High	Completed
FR-10	A valet can capture dropoff photographs per camera angle and later pickup photographs for the same angles.	High	Completed
FR-11	The system compares matched angles and produces candidate damage findings labelled as unverified potential damage.	High	Completed
FR-12	Only a manager or administrator can confirm or reject a finding; the capturing operator cannot.	High	Completed
FR-13	GPS fixes can be ingested for a custody session and are classified against the zone polygon with a stated tolerance.	High	Completed

ID	Requirement	Priority	Status
FR-14	Six simulated movement scenarios can be generated through the real ingestion and classification path, labelled as simulated.	Medium	Completed
FR-15	Geofence and movement anomalies raise alerts with severity and explanation, visible to the appropriate roles.	High	Completed
FR-16	Managers can acknowledge and resolve alerts; perrecipient read state is tracked.	Medium	Completed
FR-17	A customer can request retrieval; the request enters a queue visible to operators.	High	Completed
FR-18	The system returns a retrieval-time estimate with its contributing factors and an explicit statement that it is based on synthetic data.	Medium	Completed
FR-19	A valet can mark a vehicle ready for pickup and complete handover, closing the custody session and releasing occupancy.	High	Completed
FR-20	Every state-changing action writes an audit entry that cannot be edited or deleted through the application.	High	Completed
FR-21	Customers, operators and managers each receive a role-scoped dashboard with aggregate metrics and export.	Medium	Completed
FR-22	The system displays a live map of the zone, the recorded route and the current position, with simulated data labelled.	High	Completed
FR-23	Real GPS hardware can be attached without changing classification or presentation code.	Medium	Partial — ingestion accepts a hardware source; no device integration
FR-24	An operator can capture photographs natively from a mobile device.	Low	Planned
FR-25	Retrieval estimates are validated against real yard data.	Medium	Planned

2.4 Non-Functional Requirements

Table 2.3 states each non-functional requirement beside the mechanism in the design that addresses it.

Table 2.3: Non-functional requirements and how they are addressed

ID	Quality attribute	Requirement	Design response
NFR-01	Security — secrecy	No password, key or credential is stored in source, logs or responses.	PBKDF2-SHA256 hashing; secrets in environment variables; response serialisers exclude credentials.
NFR-02	Security — authorisation	Every privileged action is enforced on the server, not hidden in the client.	Role guards on routes; capability sets; 403/401 tests including crossaccount attempts.
NFR-03	Security — integrity	Security-relevant history cannot be rewritten.	Append-only audit table enforced by a database trigger; token revocation denylist.
NFR-04	Correctness	Invalid states and malformed input are refused with a precise error, never silently accepted.	State allow-list; field validation; consistent error envelope with machinereadable codes.
NFR-05	Traceability	Every stored verdict and estimate carries the inputs that produced it.	Verdict evidence dictionaries; per-fix source and distance; model version on predictions.
NFR-06	Honesty of AI output	No component may present a heuristic as a trained model, or a simulated figure as measured.	Classical CV labelled as such; synthetic-data flags on every payload; interface banners.
NFR-07	Usability	A first-time user can complete the primary task of their role without training.	Role-specific navigation; explicit empty, loading and error states; confirmation dialogs; toasts.
NFR-08	Accessibility	Interactive elements are reachable by keyboard and announced to assistive technology.	Semantic controls, labels, live regions for asynchronous outcomes, focus management in dialogs.
NFR-09	Performance	Interactive list and dashboard requests answer in well under a second on a developer machine.	Indexed queries, pagination, servercomposed dashboard payloads, polling rather than chatty fan-out.
NFR-10	Maintainability	Business logic is separable from transport and storage.	Routes delegate to services; AI modules import neither Flask nor the ORM; migrations are versioned.

NFR-11	Portability	The system runs on a standard Windows development machine with no cloud dependency.	Documented prerequisites; environment files; seeded data; OpenStreetMap tiles that need no API key.
NFR-12	Testability	Core behaviour is covered by automated tests that run without manual setup.	1,088 backend tests against a real PostgreSQL instance; 671 frontend tests; end-toend harness.

2.5 Hardware and Software Requirements

2.5.1 Development and demonstration environment

Table 2.4 lists the software required to build, seed and run the system; Table 2.5 lists the hardware it was developed and tested on.

Table 2.4: Software requirements

Layer	Software	Version used	Purpose
Operating system	Windows 10/11 (documented), Linux (tested)	—	Local development and demonstration
Runtime	Python	3.13	Backend, AI modules and scripts
Runtime	Node.js	20.x	Frontend build and test tooling
Database	PostgreSQL	17	Relational store for all domain data
Backend framework	Flask	3.x	REST API and request lifecycle
ORM and migrations	SQLAlchemy / Alembic	2.x / 1.x	Model mapping and schema versioning
Vision	OpenCV (opencv-python), NumPy	4.x / 2.x	Image alignment, differencing, region analysis
Machine learning	scikit-learn	1.x	Prototype classifier and retrieval estimator
Frontend framework	React, Vite	19 / 8	Single-page application and build
Mapping	Leaflet, React-Leaflet	1.9 / 5	Zone polygon, route and marker rendering
Testing	pytest, Vitest, Testing Library	—	Backend and frontend suites
Version control	Git	—	Source history; public repository

Table 2.5: Hardware requirements

Environment	Hardware	Notes
Development machine	Quad-core CPU, 8 GB RAM, 20 GB free disk	Runs PostgreSQL, Flask, Vite and the test suites concurrently.
Demonstration machine	Any modern laptop with a current browser	Backend and frontend may run on one machine; the browser connects to the frontend origin.
Server (future)	2 vCPU, 4 GB RAM, managed PostgreSQL	A production deployment is out of scope for this milestone.
Optional client	Smartphone browser	The interface is responsive; native capture is planned, not implemented.
GPS hardware (planned)	Any NMEA-capable tracker or Android device	Not integrated in this milestone; the ingestion contract is already defined.

2.6 Feasibility Study

2.6.1 Technical feasibility

Every component uses a mature, documented open-source library that runs on a standard laptop. The critical technical questions were whether a geofence can be evaluated against an irregular polygon with an explainable verdict, whether an OpenCV differencing pipeline can produce usable candidate findings from handheld photographs, and whether the three tiers can be developed and tested independently. All three were answered affirmatively during implementation: the geofence evaluates ray-casting containment and boundary distance against the stored ring, the vision pipeline aligns and normalises before differencing, and the separated tiers communicate exclusively through a documented REST contract with an automated specification check.

2.6.2 Economic feasibility

The stack is entirely open source; there are no licence fees and no paid API keys. Map imagery uses OpenStreetMap tiles, which require attribution but no subscription. The only cost is the developer's own hardware and time. For a facility, the operational case rests on avoided disputes and faster retrieval rather than on software licensing, which is why the transparency components were prioritised over convenience features.

2.6.3 Operational feasibility

The workflow mirrors existing practice rather than replacing it: a valet still takes the key and parks the car, and a customer still presents a token — now a QR code. Two additions impose real effort on staff: photographing the vehicle at drop-off and pickup, and using the dashboard instead of a paper register. Chapter 4 documents how the interface reduces that effort through pre-filled context, multi-angle capture in a single submission, and a retrieval queue that replaces verbal coordination.

2.6.4 Legal and ethical feasibility

The system records vehicle positions and photographs during a period when the owner has voluntarily surrendered the vehicle, which is a defensible basis for processing. Three design constraints follow from this and are implemented: photographs are of the vehicle, not of people; a customer can see only their own vehicle's records; and all telemetry in this milestone is synthetic, so no real person's location data is processed. The report states plainly that a production deployment would require a retention policy, a privacy notice and a lawful basis recorded under the applicable data-protection law; those are governance tasks, not code, and are listed as future work.

2.6.5 Schedule feasibility

The project was planned in three phases. Phase 1 delivered the specification and skeleton; Phase 2, reported here, delivered the functional core (eight modules) with its test suite; Phase 3 addresses hardware integration, field validation, deployment hardening and the final dissertation. Section 6.6 sets out the remaining schedule, with dates left as placeholders for the team and mentor to confirm.

2.7 Use Case Descriptions

Three use cases are described in detail because they carry the project's core guarantees: the custody handover (the moment evidence begins), the tracking view (the transparency promise), and damage verification (the accountability guarantee). The complete set of interactions is shown in the use-case diagram in Figure 3.5.

2.7.1 UC-1: Check in a vehicle and open custody

Table 2.6 specifies UC-1 in full: actors, preconditions, main flow, alternates and postconditions.

Table 2.6: Use case UC-1 — check in a vehicle

Field	Description
Actors	Valet Operator (primary); system; Customer (indirect).
Preconditions	The vehicle is registered; the operator is authenticated; a valid parking zone exists with capacity.
Trigger	The customer hands over the vehicle at the drop-off point.
Main flow	1. Operator selects the vehicle and issues a ticket. 2. System generates a ticket code, QR payload and QR PNG. 3. Operator selects the parking zone and submits check-in. 4. System validates the ticket state and role, opens a custody session at PARKED, increments zone occupancy, writes an audit entry. 5. System returns the ticket with the custody session and projected status IN_CUSTODY.

Alternate flows	A2: the vehicle already holds an unstarted ticket — the system refuses with 409 and returns the existing ticket id. A3: the zone is at capacity — check-in is refused. A4: the ticket is not in an issuable state — refused with an explanatory conflict.
Postconditions	A custody session exists with a zone and start time; occupancy is incremented in the same transaction; an audit record names the operator.
Exceptions	401 unauthenticated; 403 role not permitted; 409 state conflict; 422 validation failure.

2.7.2 UC-2: View live tracking and alerts

Table 2.7 specifies UC-2 in the same form, including the freshness limitation that polling imposes.

Table 2.7: Use case UC-2 — view live tracking

Field	Description
Actors	Customer (primary); Valet Operator and Manager (same view, broader scope).
Preconditions	The user is authenticated; a custody session exists for the vehicle; the user is entitled to see that session.
Trigger	The user opens Live Tracking for a vehicle.
Main flow	1. Client requests the tracking view for the session. 2. API authorises the request against session ownership or staff role. 3. API returns the zone (with its polygon and tolerance), the ordered track, the latest fix, per-status counts and provenance flags. 4. Client draws the polygon, tolerance band, route and markers; fits the view; displays the latest reading and alerts. 5. Client polls at a fixed interval so the view stays current.
Alternate flows	A2: no fixes yet — an explicit empty state is shown rather than a blank map. A3: map tiles unavailable — an explanatory notice is displayed while the vector overlays remain drawn from recorded data. A4: malformed coordinates — invalid points are dropped, valid ones still render.
Postconditions	No state is changed by viewing; the request is authorised and attributable.
Exceptions	403 if the session belongs to another customer; 404 if the session does not exist.

2.7.3 UC-3: Verify a damage finding

Table 2.8 specifies UC-3, the verification step that keeps an uncertain detection from being recorded as fact.

Table 2.8: Use case UC-3 — verify a damage finding

Field	Description
Actors	Valet Manager or Administrator (primary); System.
Preconditions	A pickup inspection has produced at least one candidate finding in state UNVERIFIED.
Trigger	The manager reviews the condition report.
Main flow	1. Manager opens the condition report for a custody session. 2. System returns drop-off and pickup photographs, per-angle comparisons and the findings with their reasoning. 3. Manager examines the evidence and submits CONFIRMED or REJECTED with a note. 4. System records the decision, the verifier's identity and the timestamp, and writes an audit entry.
Alternate flows	A2: the capturing operator attempts verification — refused with 403. A3: the finding is already verified — the new decision replaces the previous one with both recorded in audit.
Postconditions	The finding carries a human decision attributed to a named account.
Exceptions	403 wrong role; 404 unknown finding; 422 unknown status.

2.8 Module Descriptions

Table 2.9 describes each module, the artefacts that implement it and its implementation status.

Table 2.9: Module descriptions and implementation status

Module	Purpose	Key artefacts	Status
M1 Authentication and access control	Establish identity and enforce role-based limits.	*services/auth.py*, *utils/security.py*, *routes/auth.py*	Complete
M2 Vehicle registry	Maintain the vehicles a customer owns.	*services/vehicles.py*, *models/vehicle.py*	Complete
M3 Ticket and QR	Issue and present the digital valet ticket.	*services/tickets.py*, *services/qr_codes.py*	Complete
M4 Custody lifecycle	Open, move and close the custody session under an enforced state machine.	*services/tickets.py*, *models/custody_session.py*	Complete
M5 Condition assessment	Capture photographs, compare angles, record and verify findings.	*services/inspections.py*, *ai/vision/**	Complete
M6 Positioning and geofencing	Ingest fixes, classify against the zone, expose the track and map.	*services/geofence.py*, *ai/anomaly/geofence.py*	Complete (simulated feed)
M7 Alerts and audit	Raise, triage and preserve security and action records.	*services/alerts.py*, *services/audit*.py*	Complete
M8 Retrieval estimation	Predict waiting time and expose its basis.	*services/retrieval.py*, *ai/retrieval/**	Complete (synthetic)

M9 Dashboards and reporting	Compose role-scoped operational views and exports.	*services/*_dashboard.py*, *routes/dashboard.py*	Complete
M10 Hardware telemetry integration	Attach a real tracker to the ingestion contract.	Contract defined; device adapter absent	Planned — 40%
M11 Mobile capture	Native camera capture for operators.	Not started	Planned — 40%
M12 Deployment hardening	Containerisation, reverse proxy, backups, monitoring.	Partially documented in README	Planned — 40%

CHAPTER 3 SOFTWARE DESIGN

3.1 System Architecture Overview

The system is a three-tier web application with a deliberately framework-free analytical layer beneath it. The presentation tier is a React single-page application; the application tier is a Flask REST API; the data tier is PostgreSQL. The analytical layer contains the vision, geofence and machine-learning modules that the application tier calls into. Figure 3.1 shows the arrangement, including the external services the system depends on and the boundary of what is simulated in this milestone.

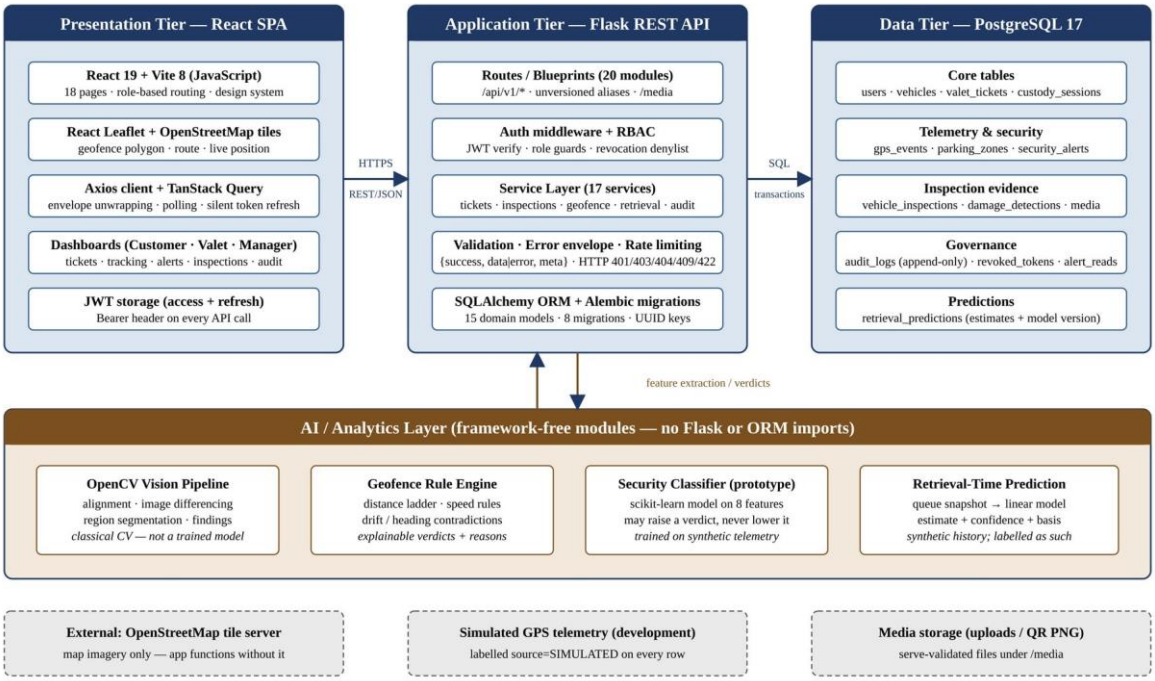


Figure 3.1: System architecture

Three properties of this architecture are load-bearing and were decided early.

- **The analytical layer cannot import Flask or SQLAlchemy.** A rule in the repository forbids it, and the test suite enforces the import graph. The consequence is that the geofence engine, the vision pipeline and both models take plain numbers and return plain results. They can therefore be tested without a database, run in a script, and reasoned about in isolation — which is exactly how the measured results in Chapter 5 were produced.

- **The browser never learns the backend's address.** In development the Vite server proxies */api* and */media* to Flask, so every request from the browser is same-origin. There is no CORS configuration to get wrong, and no backend URL is baked into the bundle.
- **The data tier enforces invariants that the application must not be trusted to remember.** Uniqueness, range checks, foreign-key behaviour and the append-only audit rule are database constraints, not service-layer conventions.

3.1.1 Layering and dependency direction

Table 3.1 states which layer may depend on which, and what enforces the restriction.

Table 3.1: Layers and their allowed dependencies

Layer	May depend on	Must not depend on	Enforcement
Routes (HTTP)	Services, utils	Models directly for business rules	Review; services own the rules
Services (business rules)	Models, utils, AI modules	Flask request objects	Review; services accept an AuthContext
AI modules	Each other, NumPy/OpenCV/scikitlearn	Flask, SQLAlchemy, models	Import-graph test in the suite
Models (persistence)	SQLAlchemy	Routes	Review
Utils (cross-cutting)	Standard library	Routes, services	Review

3.1.2 Request lifecycle

A request passes through a fixed sequence: the proxy forwards it to Flask; middleware assigns a request identifier and applies rate limits; the route decorator resolves the bearer token into an *AuthContext* and checks the required role; the route validates the payload and calls a service; the service performs the business rule inside a transaction, writes audit records, and returns a domain object; the route serialises it into the standard envelope. Errors raised anywhere in that chain are converted by a single handler into the same envelope with an appropriate status code.

3.2 Frontend–Backend Communication

Communication is REST over JSON with a single response envelope. The envelope is the most important convention in the system because it makes every response—successful or not—structurally identical, which removes a class of client-side branching.

```
{ "success": true,
  "data":    { ... domain payload ... },
  "meta":    { "request_id": "…", "server_time": 1791911235.39 } }
{ "success": false,
  "error":   { "code": "CONFLICT",
               "message": "This custody session is already closed.",
               "details": { … machine-readable context … } },
  "meta":    { "request_id": "…", "server_time": … } }
```

The client's axios instance attaches the bearer token, adds a request identifier, and unwraps *data* so components receive plain domain objects. A single error normaliser converts any failure—HTTP error, timeout, or network outage—into *{ status, code, message, details, requestId }*. Components never parse an error shape themselves, which is why the same error component works on every screen.

3.2.1 Freshness: polling, stated rather than implied

Live views refresh by polling at a fixed interval through TanStack Query. Polling was chosen over WebSockets deliberately: the data changes on the order of tens of seconds, the deployment target is a single local process, and a polling design has no reconnection state machine to get wrong. The cost is that the screen can be one interval stale, so the API reports *"delivery": "polled"* in the payload and the interface labels the freshness rather than implying real-time push.

3.2.2 Session continuity

Access tokens are short-lived; a refresh token is stored alongside. When a request fails with 401 on a domain endpoint, the client performs a single-flight refresh—concurrent failures share one refresh call—and replays the original request once. If the refresh itself is refused, the session is signed out with an explanatory banner. A transient network failure during refresh does not destroy the session; it surfaces the original error and lets the next request retry. Signing out sends the refresh token so both halves are revoked server-side.

3.3 API Architecture and Request Flow

The API exposes 63 canonical operations under */api/v1*, with unversioned aliases for the contracts the brief names explicitly. Versioning is in the path; the version is also reported by the health endpoint and embedded in the OpenAPI document. Table 3.2 summarises the conventions that every endpoint follows.

Table 3.2: API conventions applied uniformly

Concern	Convention	Rationale
Envelope	*{success, data|error, meta}*	One parsing path on the client; request id always present for support.
Status codes	200/201 success; 401 unauthenticated; 403 wrong role; 404 missing; 409 state conflict; 422 validation; 429 throttled; 413/415 upload problems	The client can react correctly without reading prose.
Errors	Machine-readable *code* plus a human message and optional *details*	Tests assert on codes, not sentences.

Identity	*Authorization: Bearer <JWT>*	Stateless verification; token id enables revocation.
Pagination	*page*, *page_size* with a documented maximum	Bounded responses; no accidental table dumps.
Timestamps	ISO-8601 with an explicit offset	No ambiguity between server and client time zones.
IDs	UUIDv4 primary keys throughout	No enumerable sequence reveals volume.
Idempotence	Duplicate issuance and repeated state changes are refused with 409 rather than silently applied	A retried request cannot create a second custody session.
Documentation	OpenAPI 3.0 generated from a catalogue and verified against the live URL map	A documented endpoint that is not served (or the reverse) fails CI.

3.3.1 Endpoint groups

Table 3.3 groups the canonical operations by resource, names a representative operation from each group and states who may call it.

Table 3.3: Endpoint groups and representative operations

Group	Representative operations	Access
Authentication	*POST /auth/register*, */auth/login*, */auth/refresh*, */auth/logout*, *GET /auth/me*, *POST /auth/users*	Public / authenticated / admin
Vehicles	*GET|POST /vehicles*, *GET|PATCH|DELETE /vehicles/{id}*, *GET /vehicles/{id}/sessions*	Owner or staff
Tickets and custody	*GET|POST /tickets*, *GET /tickets/{id}*, *POST /tickets/{id}/check-in*, */complete*, */cancel*, *PATCH /custody-sessions/{id}*	Customer (own) or staff
Inspections	*POST /inspections/dropoff*, */pickup*, *GET /inspections/{custody_id}*, *PATCH /inspections/detections/{id}/verification*	Staff; verification manager only
Positioning	*POST /gps/events*, *POST /gps/simulate*, *GET /gps/{custody_id}*, */fixes*, *GET /gps/simulate/scenarios*	Staff; read for owner
Geofences	*GET|POST /geofences*, *PATCH /geofences/{id}*	Read authenticated; write manager
Alerts	*GET /alerts*, *POST /alerts/{id}/acknowledge*, */resolve*, *GET /feed*, */feed/summary*	Role-scoped
Retrieval	*POST /retrieval/request*, *GET /retrieval/queue*, */retrieval/{custody_id}*, */model*	Customer request; staff queue
Analytics	*GET /customer/me/dashboard*, */staff/dashboard*, */manager/dashboard*, */manager/report*	Role-scoped

Audit	*GET /audit*, */audit/filters*, */audit/timeline/{custody_id}*, */audit/vehicles/{id}*	Manager and admin
System	*GET /health*, */readiness*, */ml/model*, */retrieval/model*	Public health; model info authenticated

3.4 Database Design and Entity Relationships

The schema comprises 15 domain tables created by eight sequential Alembic migrations. Primary keys are UUIDs; enumerations are stored as *VARCHAR* with *CHECK* constraints rather than native PostgreSQL enums, so that adding a value is a migration the application controls. Figure 3.2 shows the entity–relationship model for the core entities; the full table list is in Appendix B.

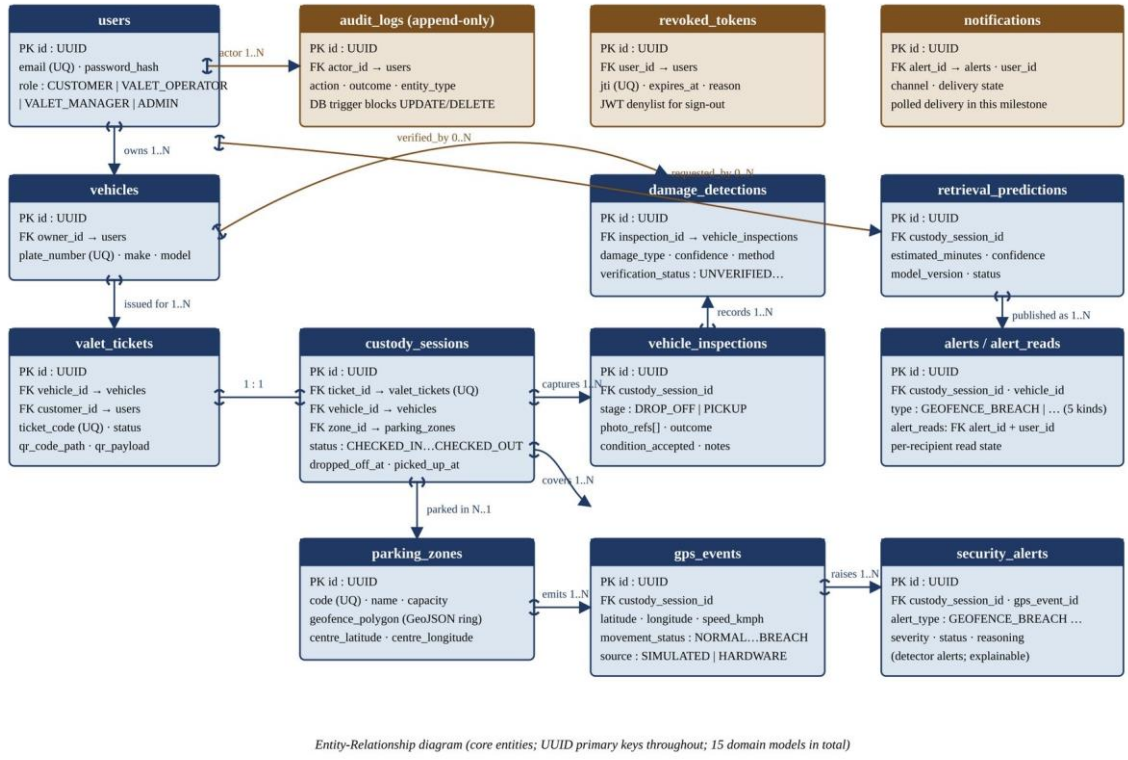


Figure 3.2: Entity–relationship diagram of the core schema

3.4.1 Design decisions in the schema

- **One custody session per ticket, one ticket per vehicle at a time.** Uniqueness constraints make a second concurrent custody impossible at the storage layer, not merely in application code.
- **Telemetry rows are immutable by intent.** *gps_events* is written once and never updated; the verdict and its evidence are part of the insert.

- **Evidence is described, not embedded.** Inspection photographs are stored as references carrying path, dimensions, byte size and SHA-256 digest, so the stored record can later prove which bytes were compared.
- **Audit is append-only by trigger.** A database trigger rejects *UPDATE* and *DELETE* on *audit_logs*; the guarantee survives a bug in the service layer.
- **Predictions are versioned.** Every retrieval estimate stores its model version, method and inputs, so an old estimate can be explained after the model changes.

Table 3.4 summarises the core entities, their purpose and the relationships between them.

Table 3.4: Core entities and their role

Entity	Purpose	Key relationships
users	Accounts, roles, status, password digest	Owns vehicles; authors audit entries
vehicles	Registered vehicles with normalised plate	Belongs to a customer; has tickets
valet_tickets	The digital ticket and its QR artefact	One vehicle, one customer; one custody session
custody_sessions	The custody window: zone, status, timestamps, acceptance	One ticket; many fixes, inspections, alerts, predictions
parking_zones	Geofence polygon, centre, radius, capacity, occupancy	Hosts many custody sessions
gps_events	One positioning fix with verdict and evidence	Belongs to a custody session
vehicle_inspections	A drop-off or pickup capture with photo references	Belongs to a custody session; has findings
damage_detections	A candidate or verified finding with reasoning	Belongs to an inspection
security_alerts	A detector alert with type, severity and reasoning	References a custody session and optionally a fix
alerts / alert_reads	Centralised feed item and perrecipient read state	References sessions and users
retrieval_predictions	Estimate, confidence, inputs and model version	Belongs to a custody session
notifications	Per-user notification records for polled delivery	References an alert and a user
audit_logs	Append-only action record	References an actor
revoked_tokens	JWT denylist for sign-out	References a user
media references	Stored descriptors for uploaded photographs and QR PNGs	Referenced by inspections and tickets

3.5 Data Flow Diagrams

Figure 3.3 presents the level-0 context diagram: the platform as a single process exchanging data with four external entities and two data stores. Figure 3.4 decomposes it into the seven processes that correspond to the implemented services.

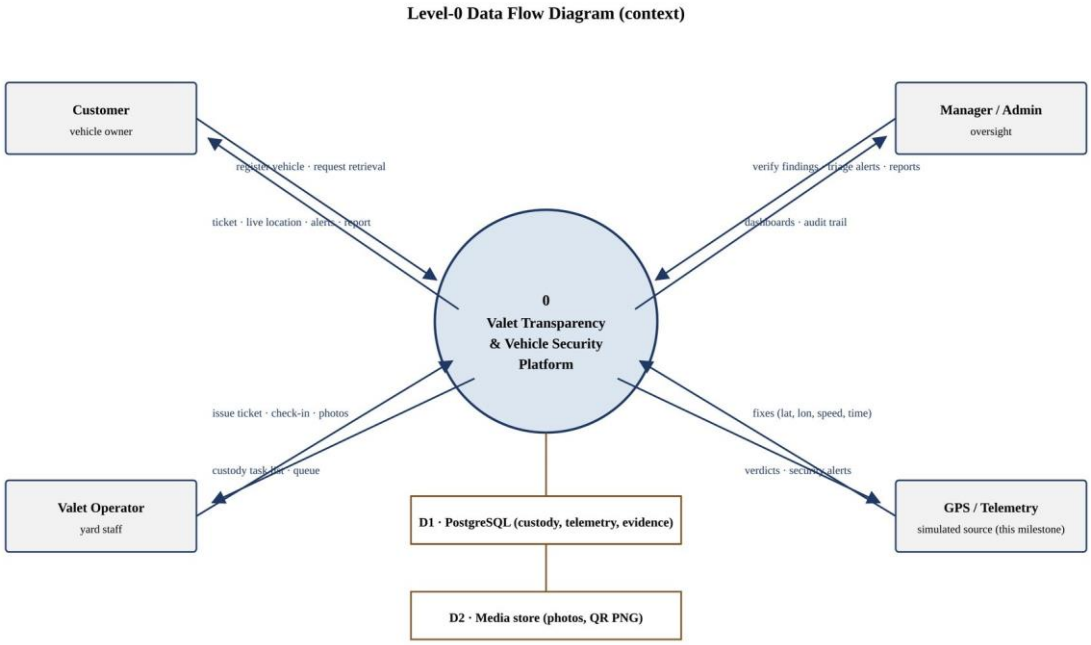


Figure 3.3: Level-0 data flow diagram (context)

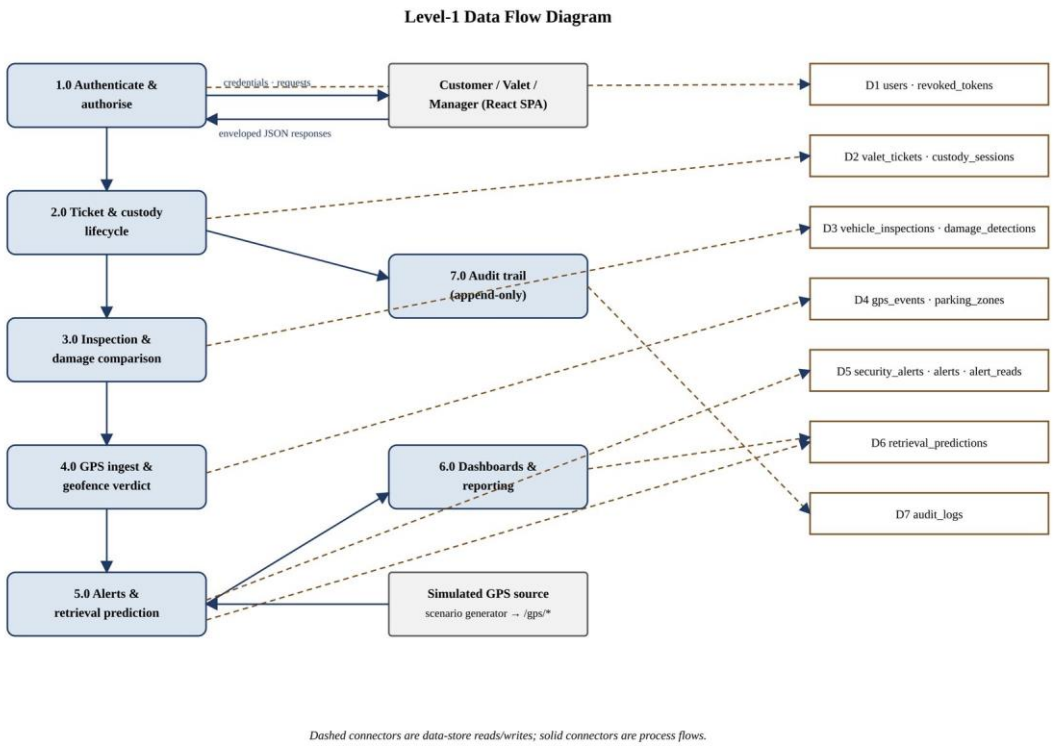


Figure 3.4: Level-1 data flow diagram

The flow that matters most is the positioning loop: the telemetry source writes fixes to D4; process 4.0 evaluates each fix against the zone stored in the same store and writes the verdict back with the fix; process 5.0 turns anomalous verdicts into alerts in D5, which the dashboards in 6.0 read. Because the verdict is stored with its inputs, the loop can be replayed and audited after the fact.

3.6 Use Case Diagram

Figure 3.5 presents the use case diagram for the four roles; each association shown is one the backend actually permits.

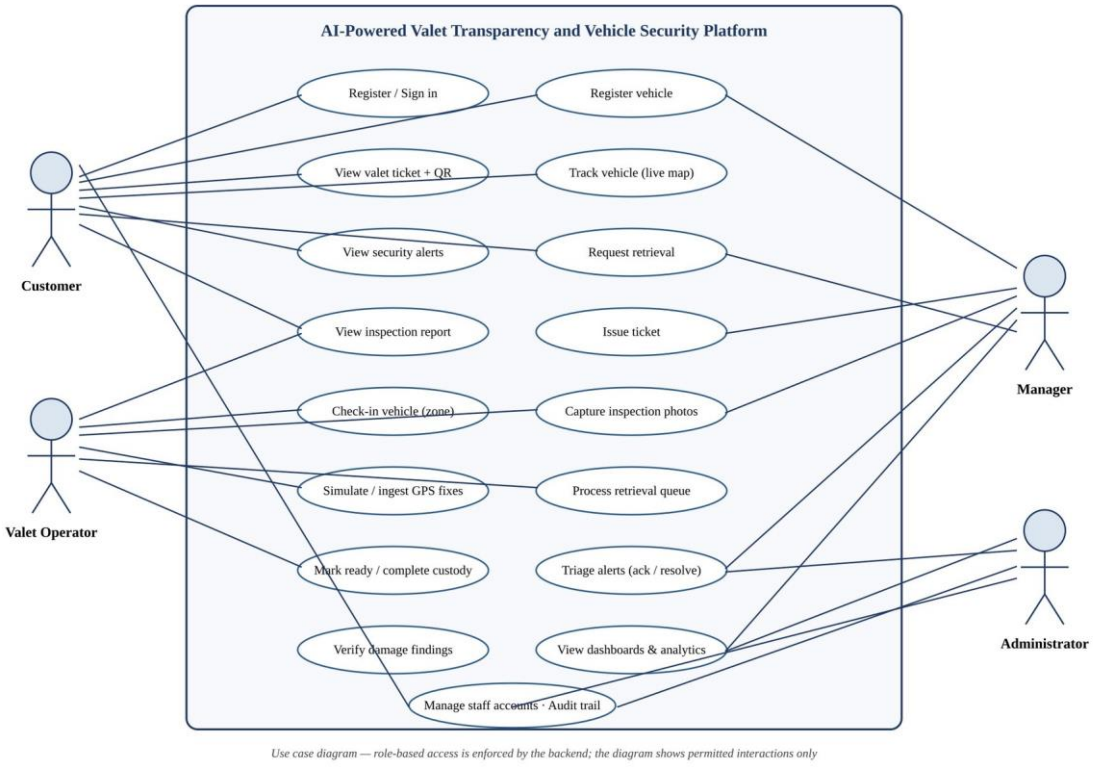


Figure 3.5: Use case diagram for the four roles

The diagram shows permitted interactions only. The backend enforces each association: a line drawn from the customer to “Verify damage findings” would be a lie, and the authorisation tests in Section 5.5 exist to keep the diagram honest.

3.7 Sequence Diagram

Figure 3.6 traces the check-in use case through the tiers. It is included because it shows the two properties the design is built around: the custody row and the zone occupancy are written in

one transaction, and the audit entry is written inside the same operation rather than as a follow-up that could fail independently.

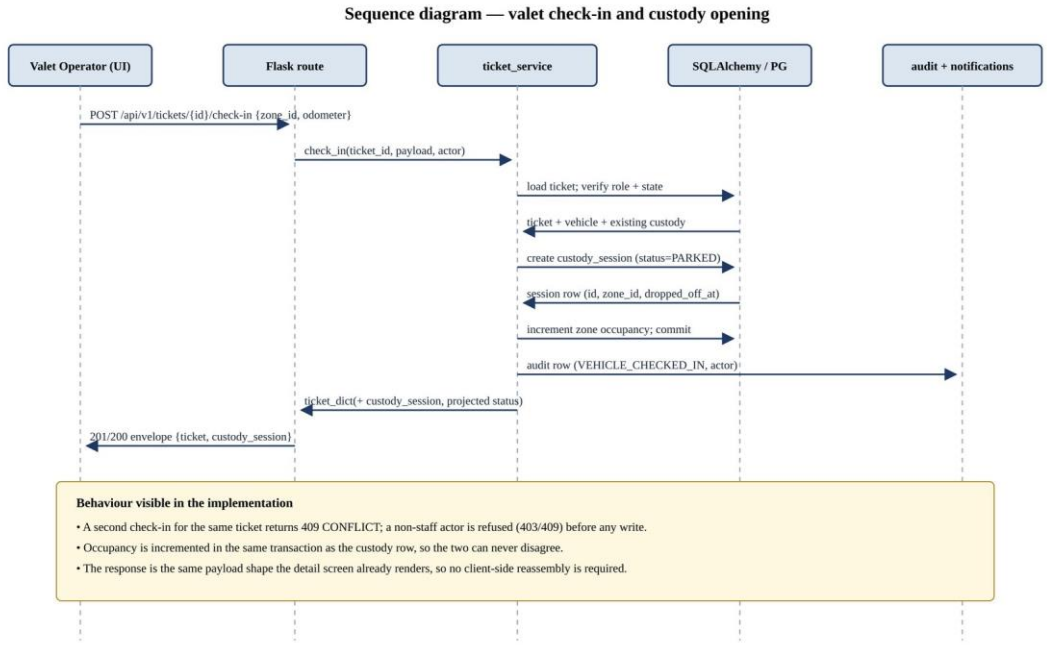


Figure 3.6: Sequence diagram for vehicle check-in

3.8 Security and Authentication Design

3.8.1 Authentication

- **Passwords** are stored as salted PBKDF2-SHA256 digests with a high iteration count. The plaintext is never logged, never returned, and never stored.
- **Tokens** are HS256 JWTs carrying subject, role, token type, unique token id, issuer, audience, issue time and expiry. Verification checks the signature, expiry, issuer, audience and token type; an unsigned (*alg: none*) token is rejected by configuration rather than by custom code.
- **Refresh tokens** are a different token type, so an access token presented to the refresh endpoint is rejected. The refresh endpoint is public because the access token it replaces may already have expired; the body token is the credential.
- **Revocation** is a denylist keyed by token id. Sign-out revokes the access token and, when the client sends it, the paired refresh token. Revocation is checked on every authenticated request.

- **Throttling** applies to failed sign-ins per email address, returning 429 with a retry window, and the check runs before the account lookup so it cannot be used to probe which addresses exist.

3.8.2 Authorisation

Authorisation is a combination of role checks and object-level ownership checks. The role check answers "may this kind of user perform this operation at all"; the ownership check answers "may this particular user act on this particular vehicle". Both are enforced in the service layer, and both return 403 rather than 404 only where revealing existence is harmless; otherwise the object is filtered from the query so it is indistinguishable from absent.

Table 3.5 states the resulting capability matrix per role; the checks that enforce it are exercised by the authorisation tests in Chapter 5.

Table 3.5: Role capability matrix (enforced by the backend)

Capability	Customer	Valet Operator	Valet Manager	Admin
Register / manage own vehicles	Yes	—	—	—
Issue a ticket	—	Yes	Yes	Yes
Check a vehicle in	—	Yes	Yes	Yes
Capture inspection photographs	—	Yes	Yes	Yes
Verify / reject damage findings	—	**No**	Yes	Yes
Ingest or simulate telemetry	—	Yes	Yes	Yes
View any vehicle's track	Own only	Yes	Yes	Yes
Acknowledge / resolve alerts	—	—	Yes	Yes
Create parking zones	—	—	Yes	Yes
View manager analytics	—	—	Yes	Yes
Read the audit trail	—	—	Yes	Yes
Create staff accounts	—	—	—	Yes

3.8.3 Additional controls

- **Upload validation.** Only permitted image types are accepted; the declared type is checked against the actual bytes; size is bounded; filenames are generated rather than trusted.
- **QR payloads.** A QR code carries a ticket reference, not a secret; scanning it resolves a ticket, it does not authenticate anyone.

- **Error hygiene.** Stack traces and database errors are never returned; the client receives a code and a curated message, and the request identifier allows an operator to find the detailed server log line.
- **Audit context.** Audit rows capture actor, role, request identifier, IP address and a reason where the operation requires one.

A security review was performed on this milestone
The project includes a written security review that reproduced and then fixed **five real vulnerabilities** before this report was prepared. The findings, their reproduction steps and their fixes are recorded in the repository. Two examples of the invariant restored: public registration can never yield a staff role, and a rejected registration leaves no partially created account behind.

3.9 GPS, Geofencing and Custody-Session Workflow

The custody lifecycle is the backbone of the product: almost every other module either feeds it or reads from it. Figure 3.7 shows the states and the permitted transitions. The transition table is an explicit allow-list in the ticket service; any move not present returns 409 with the list of permitted targets, which makes an invalid transition a diagnosable refusal rather than a silent write.

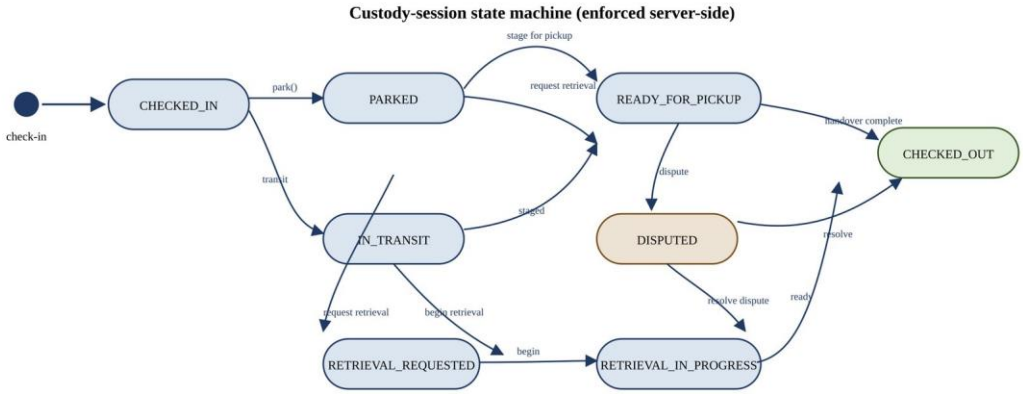


Figure 3.7: Custody-session state machine

3.9.1 Status vocabulary and projection

Table 3.6 maps each stored state to its meaning and to the status the API projects for that state.

Table 3.6: Custody states, meaning and projection

Stored status	Meaning	Ticket projection
CHECKED_IN	Session opened, vehicle not yet positioned	IN_CUSTODY
PARKED	Vehicle stationary in its assigned zone (the state check-in creates)	IN_CUSTODY
IN_TRANSIT	Vehicle moving inside the yard under a task	IN_CUSTODY
RETRIEVAL_REQUESTED	Customer has asked for the vehicle	IN_CUSTODY
RETRIEVAL_IN_PROGRESS	A valet is fetching it	IN_CUSTODY
READY_FOR_PICKUP	Staged at the handover point	READY_FOR_PICKUP
CHECKED_OUT	Handed back; session closed	COMPLETED
DISPUTED	Condition or timing contested before closure	IN_CUSTODY (flagged)

The projection exists because the brief uses a different vocabulary from the database. Internally the record keeps precise operational states; externally the ticket shows the three words a customer understands — CREATED, IN_CUSTODY, READY_FOR_PICKUP — while the API exposes both the projected and the stored value so a support engineer is never guessing.

3.9.2 The geofence verdict ladder

Each GPS fix is evaluated against the zone's stored GeoJSON ring. The distance used is the signed distance from the boundary: negative inside, positive outside. The primary rule is a distance ladder with an explicit tolerance band; further rules add speed, dwell, drift and heading-contradiction findings. The verdict is the most severe finding, and every finding carries the reason it fired.

Table 3.7 sets out the distance ladder with the configured threshold for each band and the reason the band exists.

Table 3.7: Distance rule and its configured thresholds

Condition	Verdict	Severity	Rationale
Inside the polygon (distance ≤ 0)	NORMAL	—	The vehicle is where it should be.
Outside but within the 25 m tolerance band	WARNING	Low	Consumer GPS error near a boundary is expected; it is not a breach.
Beyond the tolerance, up to the 150 m breach margin	SUSPICIOUS	Medium	Far enough outside to warrant attention, not far enough to conclude a departure.

Beyond the 150 m breach margin	BREACH	High	Consistent with the vehicle having left the permitted area.
Speed above the 20 km/h zone limit (inside only)	WARNING or SUSPICIOUS	Low / Medium	A yard is not a road; speeding inside it is a real risk. The rule does not apply outside, where road speed is expected.

Three refinements address false positives that measurement actually produced. **Movementaware escalation:** the “moving away from the yard” rule requires genuine motion, because a stationary receiver's wander otherwise points away from the zone half the time and escalated a parked car to BREACH. **Task-aware suppression:** the ”moving with no active task” rule is gated on whether the session has a retrieval in progress, so a legitimate retrieval does not read as misconduct. **Drift requiring monotonicity:** outward drift is only reported when distance increases across three consecutive fixes and by more than the tolerance band, never from a single step.

3.9.3 Scenario simulation and its boundary

Because no hardware tracker is attached in this milestone, six movement scenarios generate fixes: parked, boundary jitter, arrival, retrieval, unauthorised movement and telemetry loss. The generator is deterministic for a given seed, and its output is consumed by the **same ingestion and classification path** as a hardware fix would be — the simulator writes no verdicts. Every generated row is stored with *source* = *SIMULATED*, and both the data and the interface state that plainly.

Table 3.8 lists the six scenarios, the behaviour each generates and the verdicts it is expected to produce.

Table 3.8: Simulation scenarios and their intended verdicts

Scenario	Intended behaviour	Verdicts expected
parked	Stationary in its bay with ordinary GPS wander	NORMAL
jitter	Stationary near the boundary with a noisy receiver	NORMAL, WARNING (never a breach)
arrival	Valet drives the vehicle in from the road and parks it	NORMAL, WARNING, SUSPICIOUS
retrieval	Authorised retrieval out and back	NORMAL through SUSPICIOUS; no “no-task” finding
unauthorised	Vehicle leaves the yard at speed with no task	SUSPICIOUS, BREACH + alerts

telemetry_loss	Tracker silent for twelve minutes then resumes	NORMAL, WARNING (silence is reported)

Scenarios are run **one per custody session**. Each track is anchored to its own bay and to the current time, so running two scenarios on one session produces a jump between them that the movement rules correctly flag. This is documented in the demonstration guide rather than suppressed, because suppressing it would hide a true finding.

CHAPTER 4 IMPLEMENTATION AND USER INTERFACE

This chapter documents the implemented system module by module. Each module is described with the same structure: its purpose, the files and endpoints that implement it, the entities it reads and writes, its inputs and outputs, what is complete, and what is limited. Screenshots are taken from the running application described in this report; none is a mockup.

4.1 Technology Stack

Table 4.1 records the stack actually used by the delivered system, with the major version of each component.

Table 4.1: Implemented technology stack

Layer	Technology	Version	Role in this system
Frontend	React	19	Component model for all dashboards and forms
Frontend	Vite	8	Development server, proxy, production build
Frontend	TanStack Query	5	Server-state caching, polling, mutation lifecycle
Frontend	Leaflet / React-Leaflet	1.9 / 5	Zone polygon, route polyline, position markers
Frontend	Axios	1.x	HTTP client, envelope unwrapping, silent refresh
Backend	Flask	3.x	REST routing, blueprints, error handling
Backend	SQLAlchemy	2.x	ORM and transaction management
Backend	Alembic	1.x	Eight sequential schema migrations
Database	PostgreSQL	17	Relational store, constraints, append-only trigger
Vision	OpenCV + NumPy	4.x / 2.x	Alignment, differencing, region segmentation
ML	scikit-learn	1.x	Prototype classifier; retrieval estimator
Auth	PyJWT	2.x	HS256 sign/verify with audience and issuer claims
QR	qrcode + Pillow	—	Server-rendered PNG tickets
Testing	pytest, Vitest, Testing Library	—	1,088 backend and 671 frontend tests

4.2 Backend Implementation

The backend is organised by responsibility rather than by feature. Routes receive HTTP and do nothing else; services own the business rules and the transaction boundary; models describe persistence; utilities hold cross-cutting concerns; the AI package holds algorithms that know nothing about Flask or SQLAlchemy. Table 4.2 records the structure and the size of each part of the tree.

Table 4.2: Backend package structure

Package	Contents	Notable modules
app/	Application factory, extensions, middleware, error handlers	*create_app()*, envelope helpers, request-id middleware
routes/	20 blueprint modules; versioned registry plus unversioned aliases	*auth*, *vehicles*, *tickets*, *inspections*, *gps*, *geofences*, *alerts*, *retrieval*, *dashboard*, *staff*, *manager*, *audit*
services/	17 modules holding the business rules	*tickets* (state machine), *inspections*, *geofence*, *retrieval*, *alerts*, *audit*, three dashboards
models/	15 domain models plus shared mixins and enums	*custody_session*, *gps_event*, *vehicle_inspection*, *damage_detection*, *audit_log*
ai/	Vision, anomaly, ML and syntheticdata packages	*vision/** (7 modules), *anomaly/geofence.py*, *ml/**, *retrieval/**, *synthetic/**
migrations/	Eight Alembic revisions	*0001* foundation → *0008* audit lifecycle
utils/	Validation, security, pagination, media, rate limiting	*security.py*, *validation.py*, *responses.py*
database/	Curated demonstration seed and bootstrap	*seed.py*
tests/	30 test modules	Endpoints, engine, AI, security audit, API-documentation conformance

4.2.1 Conventions that keep the backend consistent

- **Services commit; routes never do.** A route that committed would split a business operation across two transaction owners, and the audit entry could then disagree with the change it describes.
- **Validation is explicit and early.** Required fields, formats, ranges and enumerations are checked before a service is entered, and failures raise a typed exception carrying the offending field.
- **Errors are typed and mapped once.** Nine exception classes map to nine status codes in a single handler; a new failure mode cannot accidentally become a 500.
- **Every consequential action is audited.** The audit helper is called inside the service operation, so an action cannot be performed without its record.

4.2.2 Example: the check-in service in outline

```
def check_in(ticket_id, payload, actor):
    _require_role(actor, VALET_ROLES)            # 403 if not staff
ticket = _load_ticket(ticket_id)             # 404 if absent
    _assert_issuable(ticket)                     # 409 if already started
zone = _load_zone(payload['zone_id'])        # 404 / 409 if full
session = CustodySession(ticket=ticket, zone=zone,
status=CustodyStatus.PARKED, ...)     db.session.add(session)
    zone.current_occupancy += 1                  # same transaction
_audit(AuditAction.VEHICLE_CHECKED_IN, actor=actor, ...)
    db.session.commit()                          # one commit, all or nothing
return ticket
```

The shape is repeated across services: role check, load, state check, mutate related rows, audit, single commit, return a domain object that the route serialises.

4.3 Frontend Implementation

The frontend is a single-page application with 18 page components and a small design system. Navigation is role-driven: the same build renders a customer's five-item menu or a manager's oversight menu, and each route additionally declares the roles permitted to see it.

Hiding a route is a usability measure only; the backend remains the authority.

Table 4.3 lists the implemented routes by screen and the audience each route serves.

Table 4.3: Implemented routes and audience

Route	Screen	Audience
/login, */register*	Sign in; customer registration	Public
/status	System status and dependency health	Authenticated
/dashboard	Customer dashboard: active custody, vehicle, alerts, retrieval	Customer
/vehicles	Vehicle list and registration	Customer
/tickets, */tickets/:id*	Ticket list; ticket detail with QR and timeline	All authenticated
/tracking, */tracking/:custodyId*	Session picker; live tracking with map	All authenticated (scoped)
/alerts	Security alerts with triage controls	All authenticated (scoped)
/valet	Operator dashboard: active vehicles, pickups, queue	Staff
/checkin	Issue ticket and check a vehicle in	Staff
/condition	Inspection capture for drop-off or pickup	Staff
/condition/:custodyId	Condition report with evidence and verification	Staff
/manager	Manager dashboard with filters and export	Manager, Admin
/audit	Audit trail with filters	Manager, Admin

4.3.1 Design system

A small component library (*components/ui*) provides the badge, panel, field, data row, spinner, alert banner, error notice, button, confirmation dialog, skeleton and empty state used on every screen. Fifteen stylesheets are loaded in a fixed order from the entry point; the ordering is deliberate because later sheets build on tokens defined earlier. Every important action produces feedback: a toast on success, an inline error with a request id on failure, a confirmation dialog for destructive actions, and an explicit empty or loading state rather than a blank panel.

4.3.2 Honesty labels built into the interface

- A banner on the tracking screen states that the telemetry is simulated and must not be presented as hardware data.
- The map carries a persistent “Simulated telemetry (demo)” badge.
- The manager dashboard states that the dataset contains synthetic history and marks those rows.
- The retrieval estimate is shown with its basis and a notice that the model is trained on synthetic data.
- Damage findings are labelled “Potential damage” until a manager verifies them.

4.4 Authentication and Role-Based Access

Purpose. Establish who is acting, and limit what they can do. **Implementation.** Registration creates a customer account only; staff accounts are created by an administrator through a role-restricted endpoint. Sign-in verifies the password against a PBKDF2 digest and returns a token pair. The client stores both, attaches the access token to every request, and refreshes silently when it expires.

Table 4.4 summarises the authentication module: its files, its endpoints, the controls it applies and the limits it accepts.

Table 4.4: Authentication module summary

Aspect	Detail
Files	*services/auth.py*, *utils/security.py*, *routes/auth.py*, *utils/rate_limit.py*; client: *context/AuthContext.jsx*, *services/apiClient.js*
Endpoints	*POST /auth/register*, */auth/login*, */auth/refresh*, */auth/logout*; *GET /auth/me*; *POST /auth/users* (admin)
Entities	*users*, *revoked_tokens*, *audit_logs*
Input	Credentials on sign-in; profile fields on registration; bearer token on protected calls

Output	Access and refresh tokens, the account profile, and a role that drives navigation
Completed	Registration, login, refresh, revocation, role enforcement, throttling, audit of sign-in outcomes
Limitations	Tokens are held in browser local storage; refresh tokens are not rotated. Both are documented as accepted risks with the reasoning in *docs/SECURITY-AUDIT.md*.

Figure 4.1 shows the sign-in screen, including the notice displayed when a session has been rejected.

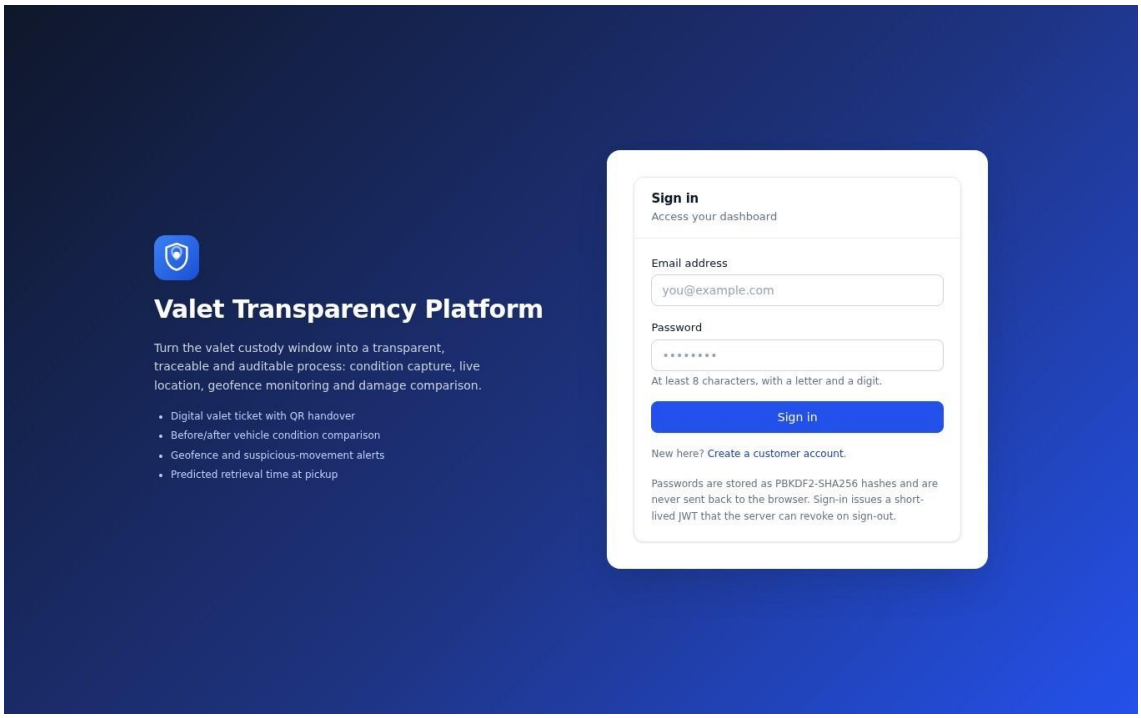


Figure 4.1: Sign-in screen with the session-rejection notice used after an expiry

4.5 Vehicle Registration and Valet Ticket Workflow

Purpose. Replace the paper token with a verifiable digital artefact, and give the owner a durable record that the vehicle was accepted. **Workflow.** A customer registers a vehicle once. At drop-off a valet issues one ticket; the system generates a unique ticket code, a QR payload and a PNG rendered on the server. Duplicate issuance for a vehicle already holding an unstarted ticket is refused with the existing ticket id so the operator can continue that job instead of creating a second one.

Table 4.5 summarises the ticket module in the same form.

Table 4.5: Ticket module summary

Aspect	Detail
Files	*services/tickets.py*, *services/qr_codes.py*, *routes/tickets.py*; client: *pages/TicketListPage.jsx*, *pages/TicketDetailPage.jsx*
Endpoints	*GET\POST /tickets*, *GET /tickets/{id}*, *POST /tickets/{id}/check-in*, */complete*, */cancel*
Entities	*valet_tickets*, *vehicles*, *custody_sessions*
Input	Vehicle id; zone id and odometer on check-in; odometer and acceptance on completion
Output	Ticket with code, QR path and URL, projected status, custody session and parking zone
Completed	Issuance, QR generation and retrieval, duplicate prevention, state projection, cancellation
Limitations	QR payload identifies the ticket but is not a secret; scanning is a lookup, not an authentication.

Figures 4.2 and 4.3 show the customer vehicle list and the digital ticket produced by a checkin, including the server-rendered QR code.

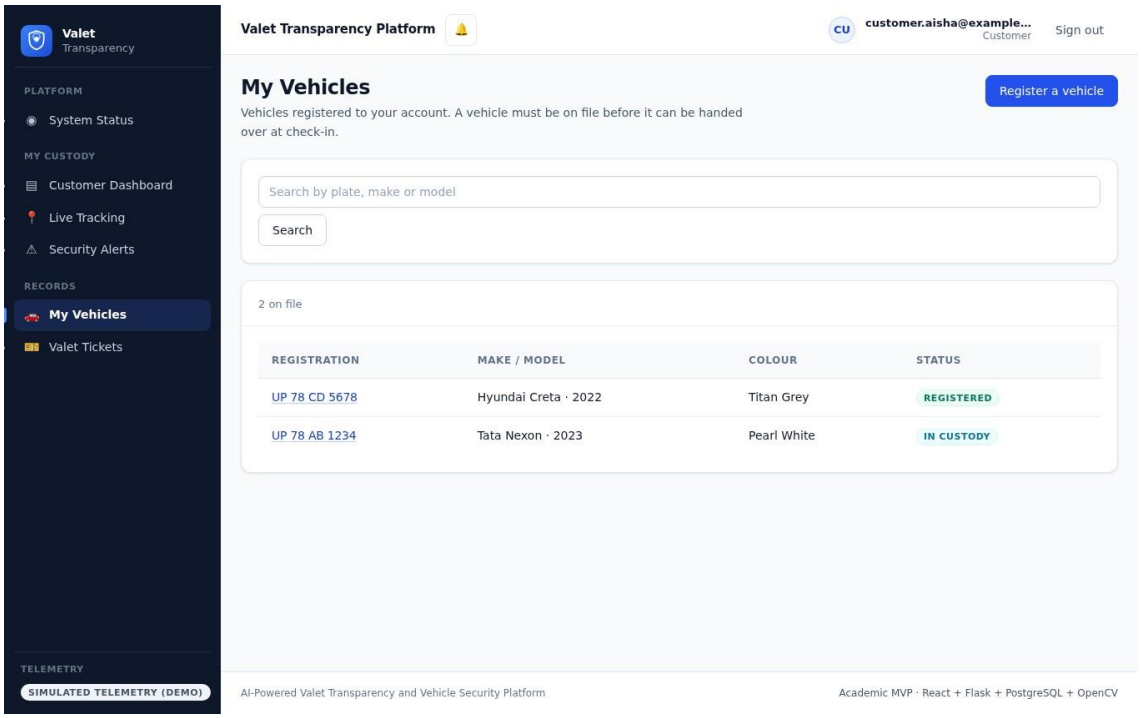


Figure 4.2: Customer vehicle list with registration entry point

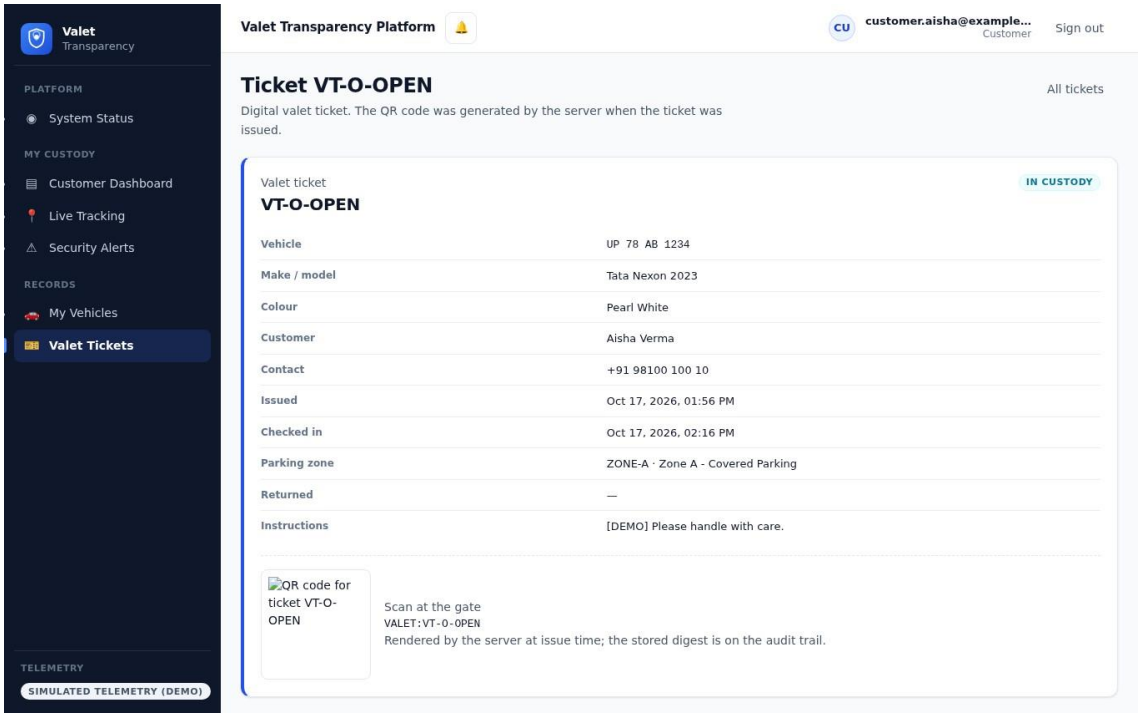


Figure 4.3: Digital valet ticket with server-rendered QR and custody summary

4.6 Vehicle Custody and Inspection Workflow

Purpose. Record the vehicle's condition at handover and at return, and compare the two, so that damage can be attributed rather than argued about. **Workflow.** The valet photographs the vehicle at drop-off, selecting a camera angle per photograph; at pickup the same angles are captured and each pair is compared. Findings appear as “potential damage“ with a confidence and the reasoning that produced them, and remain unverified until a manager confirms or rejects them.

Table 4.6 summarises the inspection module in the same form.

Table 4.6: Inspection module summary

Aspect	Detail
Files	*services/inspections.py*, *ai/vision/{alignment,preprocessing,compare,damage,regions,report,pipeline}.py*; client: *pages/InspectionCapturePage.jsx*, *pages/ConditionReportPage.jsx*
Endpoints	*POST /inspections/dropoff*, *POST /inspections/pickup*, *GET /inspections/{custody_id}*, *PATCH /inspections/detections/{id}/verification*
Entities	*vehicle_inspections*, *damage_detections*, media references
Input	Custody session id, photographs with angle labels, optional notes and odometer; decision and note on verification
Aspect	**Detail**

Output	Inspection record with photo descriptors (path, size, sha-256), per-angle comparisons, findings and a verification state
Completed	Multi-angle capture, matching by angle, comparison, evidence rendering, manager-only verification, audit of each decision
Limitations	Classical differencing, not a trained damage classifier; accuracy depends on comparable framing and lighting; unverified findings require human judgement.

Figure 4.4 shows the check-in form with zone selection and odometer capture; Figures 4.5 and 4.6 continue the same flow through inspection capture and the condition report.

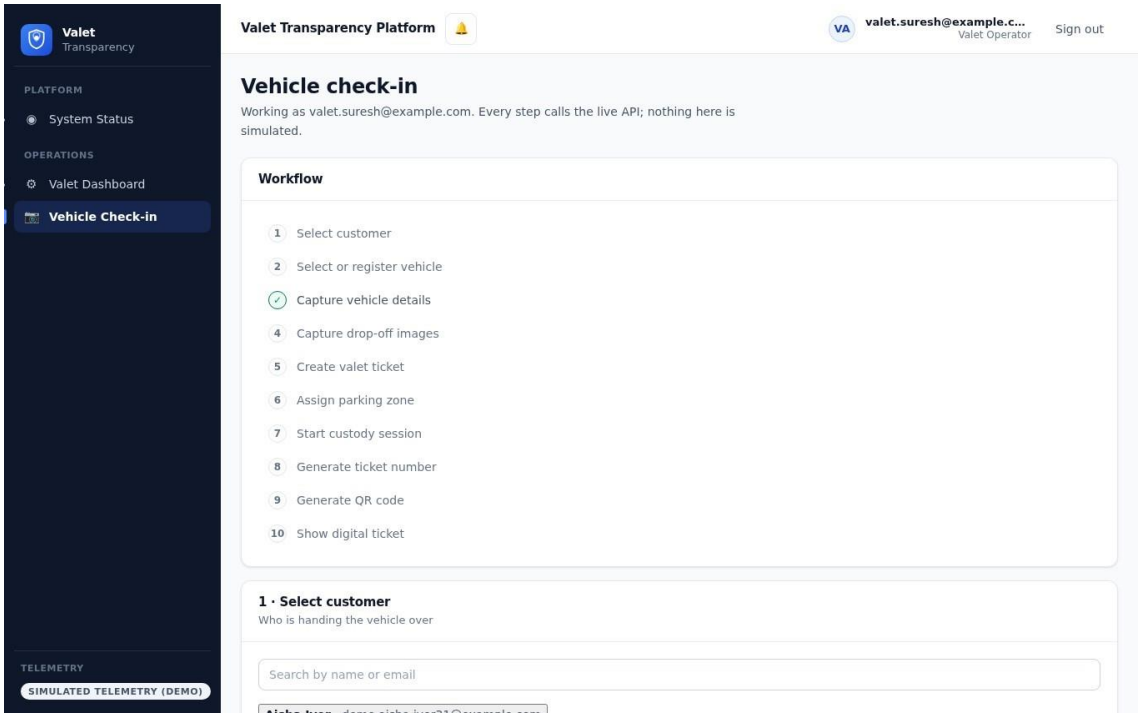


Figure 4.4: Vehicle check-in — zone selection and odometer capture

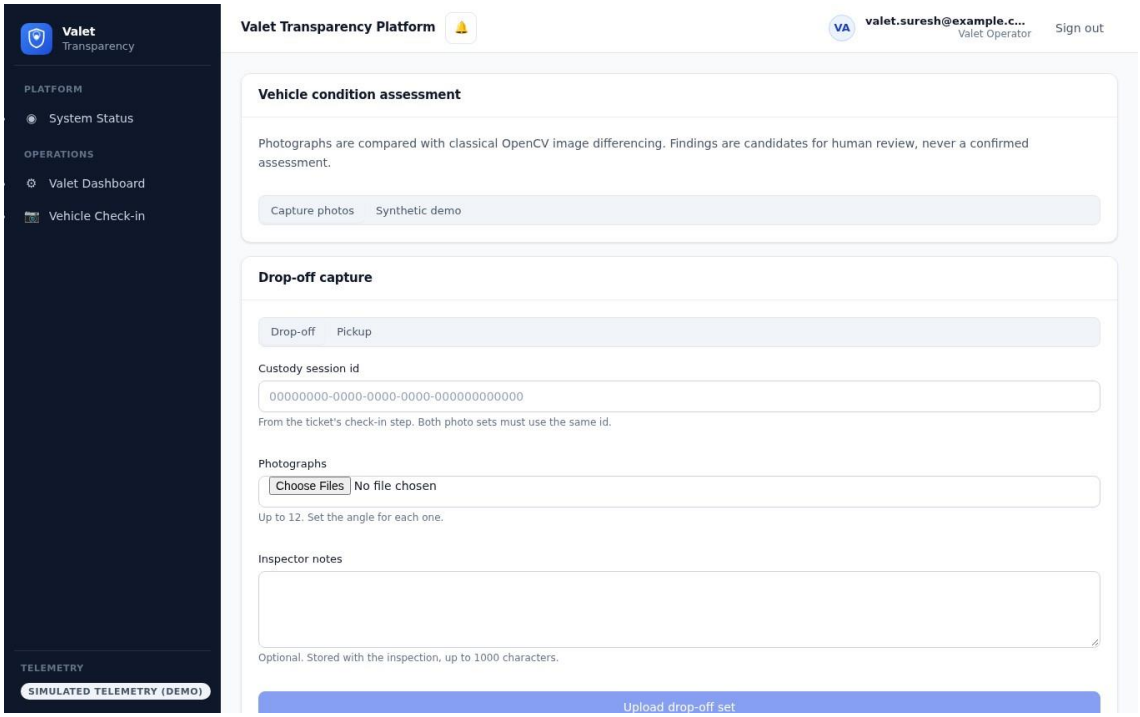


Figure 4.5: Inspection capture screen for drop-off photographs

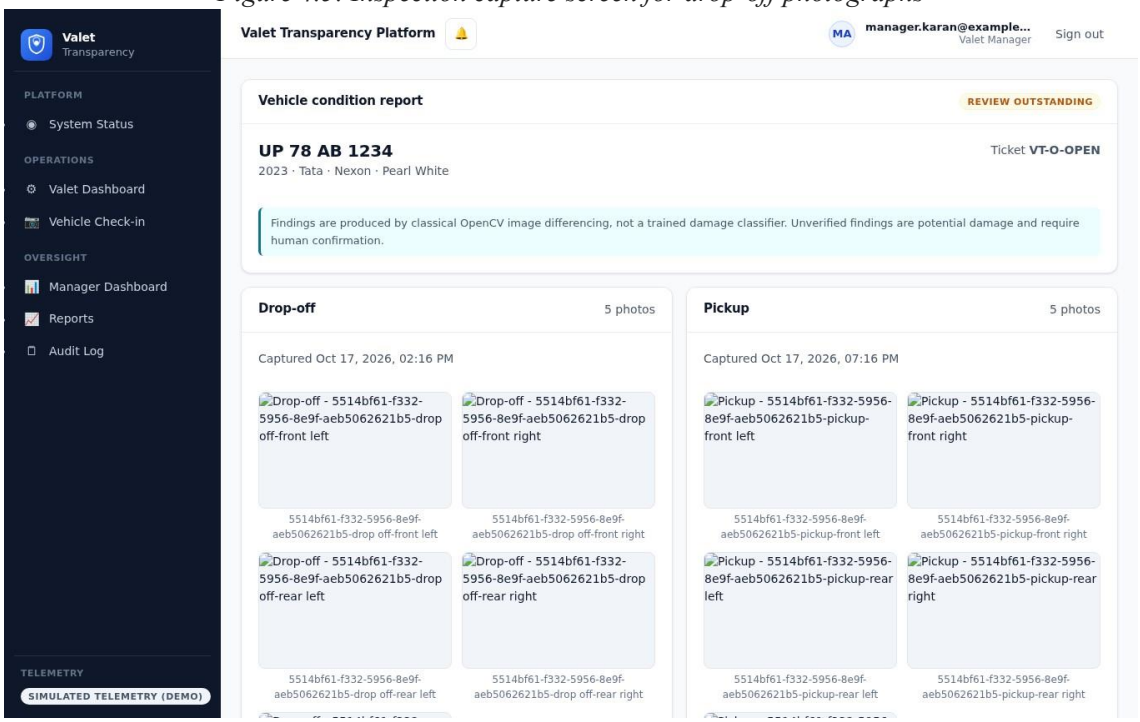


Figure 4.6: Condition report with before/after evidence and verification controls

4.7 Computer Vision and Damage-Detection Module

Purpose. Produce candidate damage findings from a drop-off/pickup photograph pair, with enough evidence for a human to decide. **Implementation.** The pipeline contains seven modules. Photographs are validated and stored with a digest; the pickup image is aligned to the drop-off baseline (feature-based alignment with a fallback to identity); illumination is

normalised; the images are differenced; regions are extracted with morphological filtering and size constraints; each surviving region is classified into a damage type from its shape and contrast characteristics and scored; and a comparison image is rendered that highlights each candidate.

Table 4.7 lists the seven pipeline stages in order, with the module that implements each and what it does.

Table 4.7: Vision pipeline stages

Stage	Module	What it does
Validation	*preprocessing.py*	Checks type, dimensions and decodability; rejects unusable uploads with a precise error
Alignment	*alignment.py*	Estimates a transform between the pair so that camera shift is not mistaken for damage
Normalisation	*preprocessing.py*	Reduces exposure and colour differences between the two capture sessions
Differencing	*compare.py*	Computes and thresholds the difference, returning statistics and parameters used
Region extraction	*regions.py*	Groups difference pixels into candidate regions above a minimum area
Classification	*damage.py*	Assigns a type (scratch, dent, paint chip …) and a confidence from measurable region features
Evidence rendering	*report.py*	Produces the annotated comparison image and the finding reasoning shown to the user

The implementation stores the parameters it used, so a finding can be explained after the fact. In the seeded dataset, representative unverified findings carry confidences such as 0.960 (dent), 0.955 (scratch) and 0.949 (paint chip), each with *method* = *CLASSICAL_CV* and *model_version* = *NULL* — the null is deliberate, because there is no trained model to attribute the finding to.

What this module does not claim
It is **classical computer vision, not a trained damage detector**. There is no learned model, no annotated dataset and no measured detection accuracy.
Confidence expresses how strongly the measured features match the configured thresholds. It is not a probability that damage exists.
Every finding is presented as *potential* damage and requires human verification, which is why verification is a separate, role-restricted act.

4.8 GPS Tracking, Map and Geofencing Module

Purpose. Show where the vehicle is, whether it is where it should be, and what happened if not. **Implementation.** Fixes enter through one ingestion endpoint, are evaluated against the

zone polygon by the rule engine, and are stored with their verdict and evidence. The tracking endpoint returns the zone (including its GeoJSON ring and tolerance), the ordered track, the latest fix, per-status counts and provenance flags. The client draws the boundary, the tolerance band, the route and markers coloured by verdict, fits the view to the zone and route, and refreshes on a fixed interval.

Table 4.8 summarises the tracking module in the same form as the modules above.

Table 4.8: Tracking module summary

Aspect	Detail
Files	*services/geofence.py*, *ai/anomaly/geofence.py*, *ai/synthetic/gps_tracks.py*; client: *pages/TrackingPage.jsx*, *components/tracking/TrackingMap.jsx*, *utils/mapData.js*
Endpoints	*POST /gps/events*, *POST /gps/simulate*, *GET /gps/simulate/scenarios*, *GET /gps/{custody_id}*, *GET /gps/{custody_id}/fixes*
Entities	*gps_events*, *parking_zones*, *custody_sessions*, *security_alerts*
Input	Fix payload (session, latitude, longitude, speed, heading, timestamp, source) or a scenario name with a seed
Output	Stored fix with verdict, distance from boundary, inside/outside flag, alert references; a tracking view for the map
Completed	Ingestion, classification, simulation of six scenarios, map rendering, provenance labelling, alerts on anomalies
Limitations	All telemetry is simulated; tile imagery depends on an external server (the map degrades to vector overlays with a notice if tiles fail).

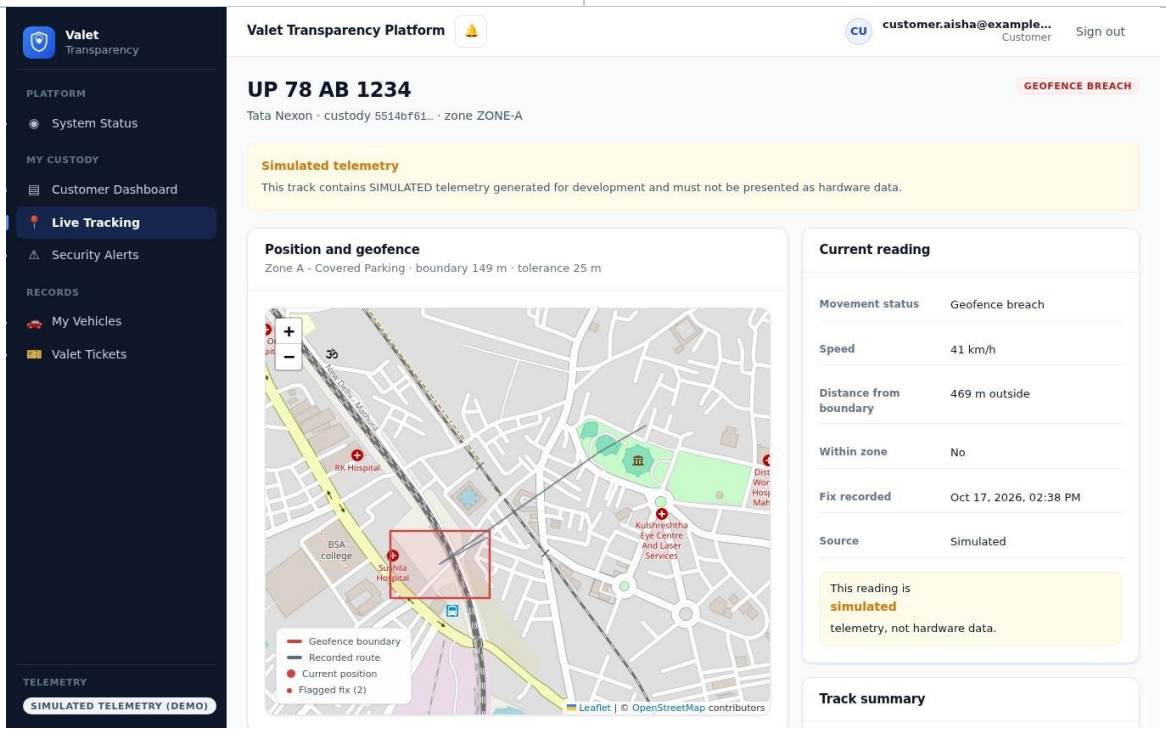


Figure 4.7: Live tracking — geofence polygon, route, position markers and legend

Figure 4.7 is worth examining closely because it is the report's main claim made visible: the zone polygon is drawn from the stored ring (not from an approximate circle); the route is the recorded track; the current position is coloured by the verdict (red for breach); the legend explains each symbol; and the banner states that the telemetry is simulated. The distance read-out — "469 m outside boundary" — is the same number the rule engine used to decide the verdict.

4.8.1 Map rendering: a defect found and fixed during this milestone

An early build showed a grey rectangle where the map should be. The cause was not missing data: the mapping library renders a container with no intrinsic height, the stylesheet that ships with it sets no height, and no application rule gave the map a height — so the map initialised into a zero-height box and only the surrounding card's background was visible. The fix was a single stylesheet rule giving the map container full height, followed by hardening: bounds fitting on first render and on vehicle change, re-measurement on resize, a visible notice when tiles cannot load, a legend, and a conversion layer that drops malformed coordinates instead of throwing inside render. Sixteen component tests and twenty conversion tests now cover these cases.

4.9 Retrieval-Time Prediction Module

Purpose. Replace a verbal "ten minutes" with an estimate derived from measured queue state, and show what the estimate is based on. **Implementation.** When a customer requests retrieval, the service measures the queue (how many requests are waiting, zone occupancy, time of day, whether the vehicle is flagged) and feeds those features to a ridge-regularised linear model. The response contains the estimate, a confidence, the contributing factors in minutes, the model version and an explicit statement that the model was trained on synthetic data.

Table 4.9 summarises the retrieval module in the same form.

Table 4.9: Retrieval module summary

Aspect	Detail
Files	*services/retrieval.py*, *ai/retrieval/{dataset,features,model}.py*, *utils/scripts/train_retrieval_model.py*
Endpoints	*POST /retrieval/request*, *GET /retrieval/queue*, *GET /retrieval/{custody_id}*, *GET /retrieval/model*, *POST /retrieval/predict*
Entities	*retrieval_predictions*, *custody_sessions*

Input	Custody session id; a queue snapshot computed at request time
Output	Estimated minutes, interval, confidence, per-factor contributions, model version and data-basis note
Completed	Queue integration, estimate with explanation, persistence of each prediction, baseline comparison
Limitations	Trained and evaluated entirely on synthetic data; the confidence is a typicality measure, not a calibrated probability.

4.10 Database Integration

Persistence uses SQLAlchemy with explicit transaction boundaries in services and Alembic for versioned schema changes. Eight migrations build the schema in dependency order, and the test suite applies them to a fresh database so that migration drift fails a test rather than surfacing in a demonstration.

Table 4.10 lists the eight migrations in the order they are applied and the scope of each.

Table 4.10: Migration sequence

Revision	Scope
0001	Foundation: users, roles, authentication support
0002	Core domain: vehicles, tickets, custody sessions, zones, alerts, audit
0003	Positioning: GPS events and zone geometry
0004	Inspections and damage detections with media references
0005	Retrieval predictions and queue support
0006	Centralised alerts, per-recipient read state, notifications
0007	Constraints and indexes for reporting queries
0008	Audit lifecycle: append-only enforcement and retiredaction support

The audit guarantee deserves its own sentence because it is enforced twice: the service layer never issues an update or delete against *audit_logs*, and a database trigger rejects such a statement if some future code path attempts it. A test asserts that the trigger fires.

4.11 Current Implementation Status

Table 4.11 records the status of each module with the evidence that supports the claim. The percentages are the weighting used in Section 6.2 to justify the sixty per cent milestone.

Table 4.11: Module status and supporting evidence

Module	Status	Evidence
M1 Authentication and RBAC	100%	Endpoint tests for every role; crossaccount isolation; 5 vulnerabilities found and fixed in review

M2 Vehicle registry	100%	Create/list/edit/retire with ownership tests
M3 Ticket and QR	100%	Issuance, duplicate refusal, QR PNG fetchable and a valid PNG
M4 Custody lifecycle	100%	State-machine tests including every illegal transition (409)
M5 Condition assessment	100%	Vision unit tests, pipeline integration, verification role tests, condition report
M6 Positioning and geofencing	95%	Engine tests with measured verdict matrices; six scenarios; map component and conversion tests. Hardware feed absent by design.
M7 Alerts and audit	100%	Alert lifecycle tests; append-only trigger test; audit timeline and filters
M8 Retrieval estimation	90%	Model metrics against baselines; queue integration; honesty fields asserted in tests
M9 Dashboards and reporting	100%	Aggregate payload tests; CSV/JSON export; role scoping
M10 Hardware telemetry	15%	Ingestion contract and source abstraction defined; no device integration
M11 Mobile capture	0%	Not started
M12 Deployment hardening	25%	Documented local deployment; no containers, proxy or backups yet

4.11.1 Interface evidence across roles

Figures 4.8, 4.9 and 4.10 show the three role dashboards, Figures 4.11 and 4.12 show the alerts and audit screens they link to, and Figure 4.13 shows the system-status view.

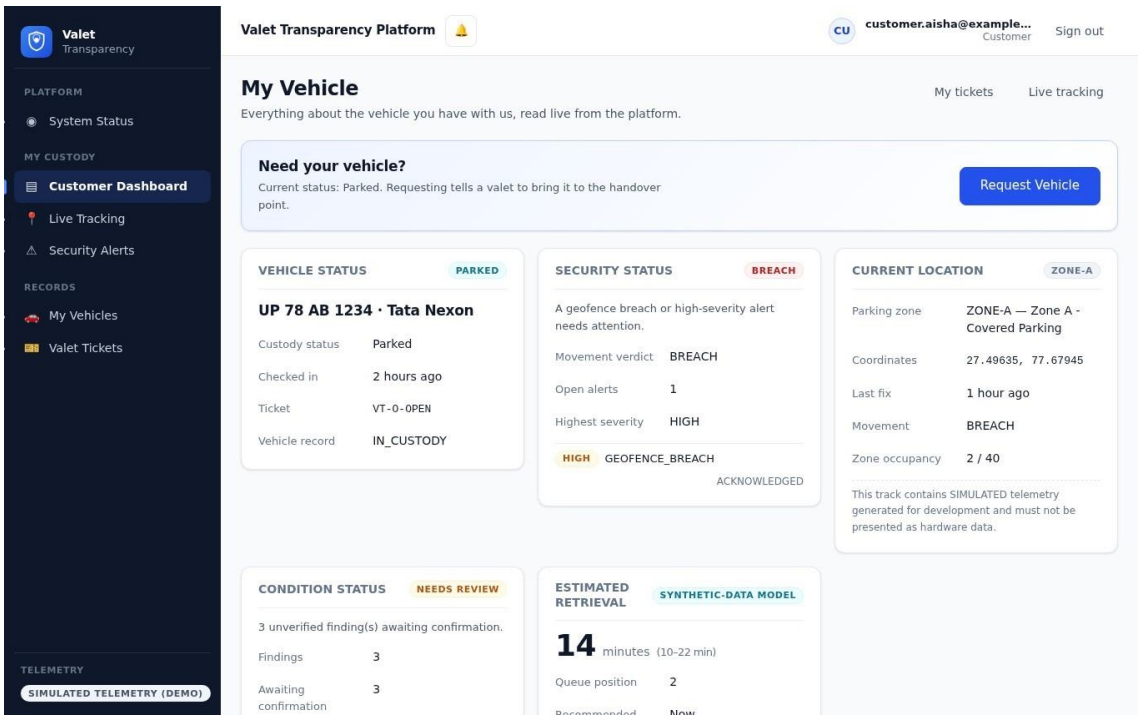


Figure 4.8: Customer dashboard — active custody, vehicle and retrieval state

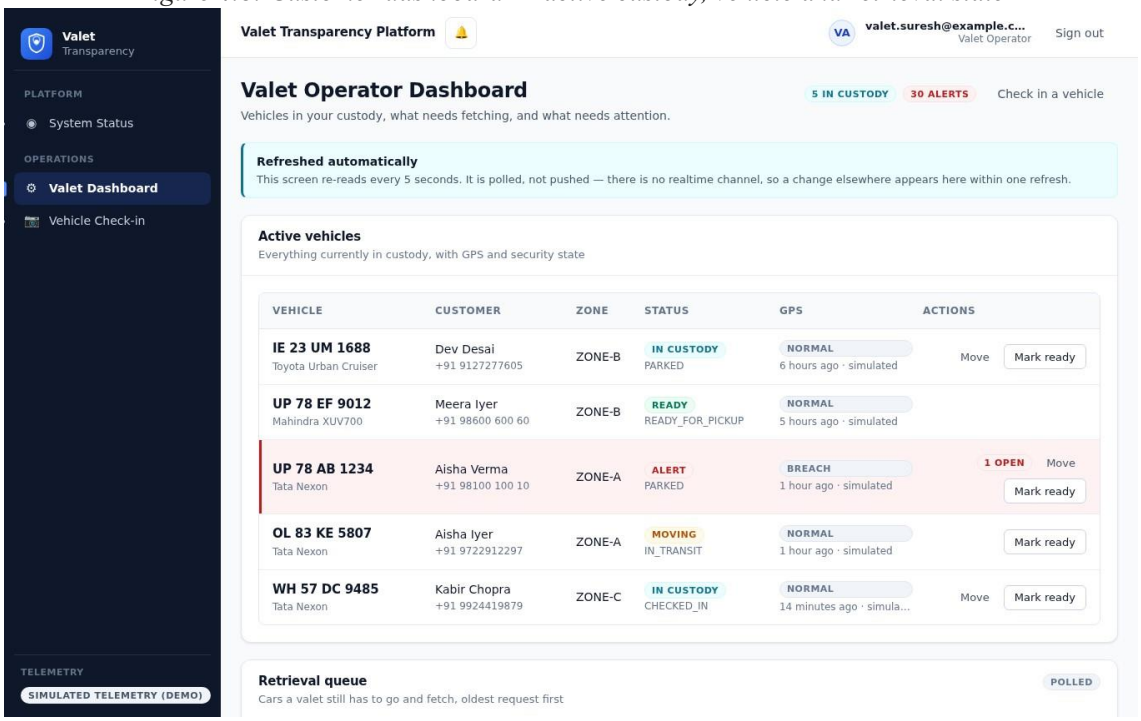


Figure 4.9: Valet operator dashboard — active vehicles, pending pickups and queue

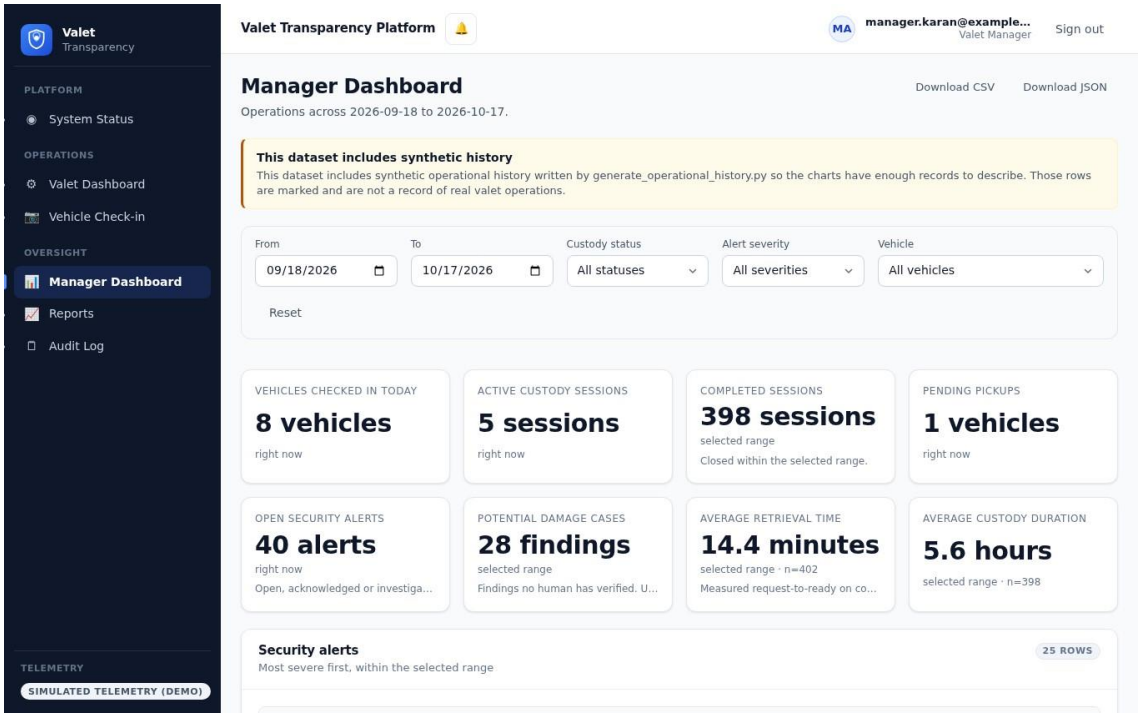


Figure 4.10: Manager dashboard — filtered metrics, alerts and export

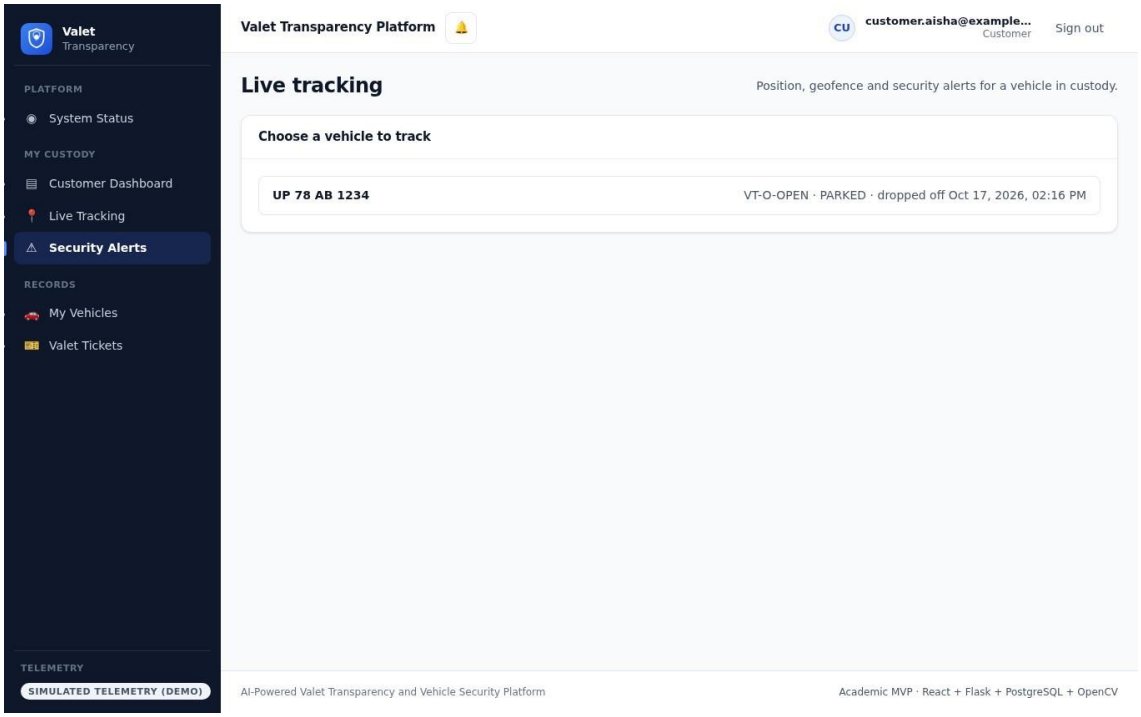


Figure 4.11: Security alerts with severity, source and triage controls

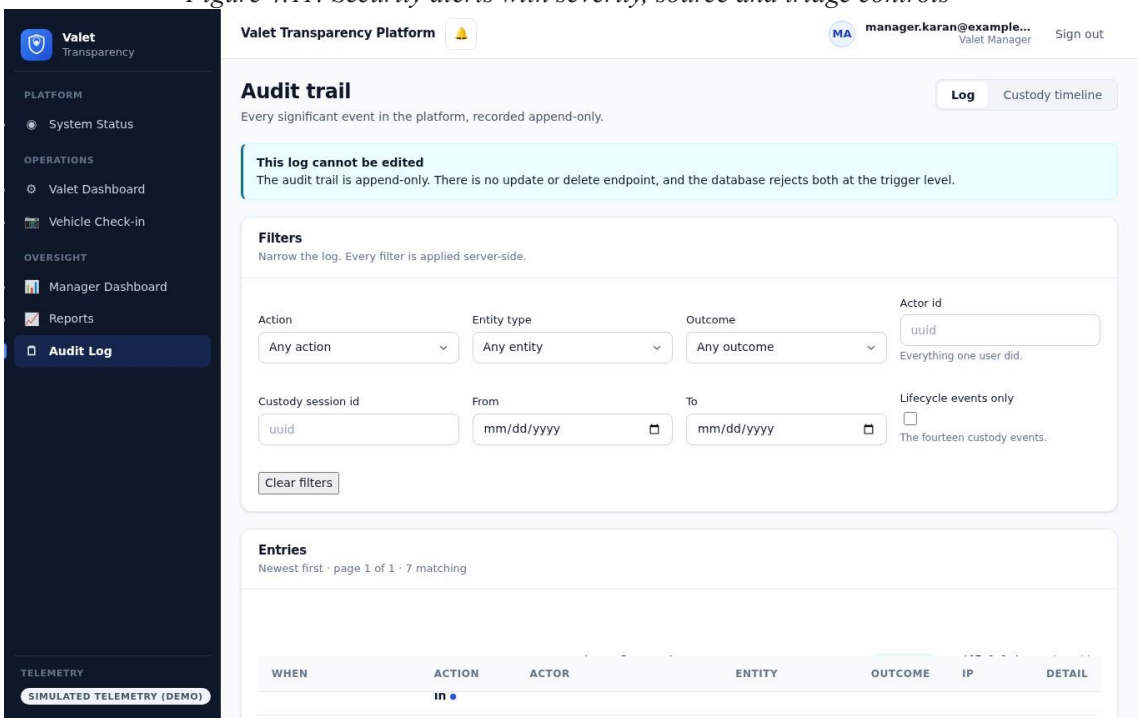


Figure 4.12: Audit trail with actor, action and entity details

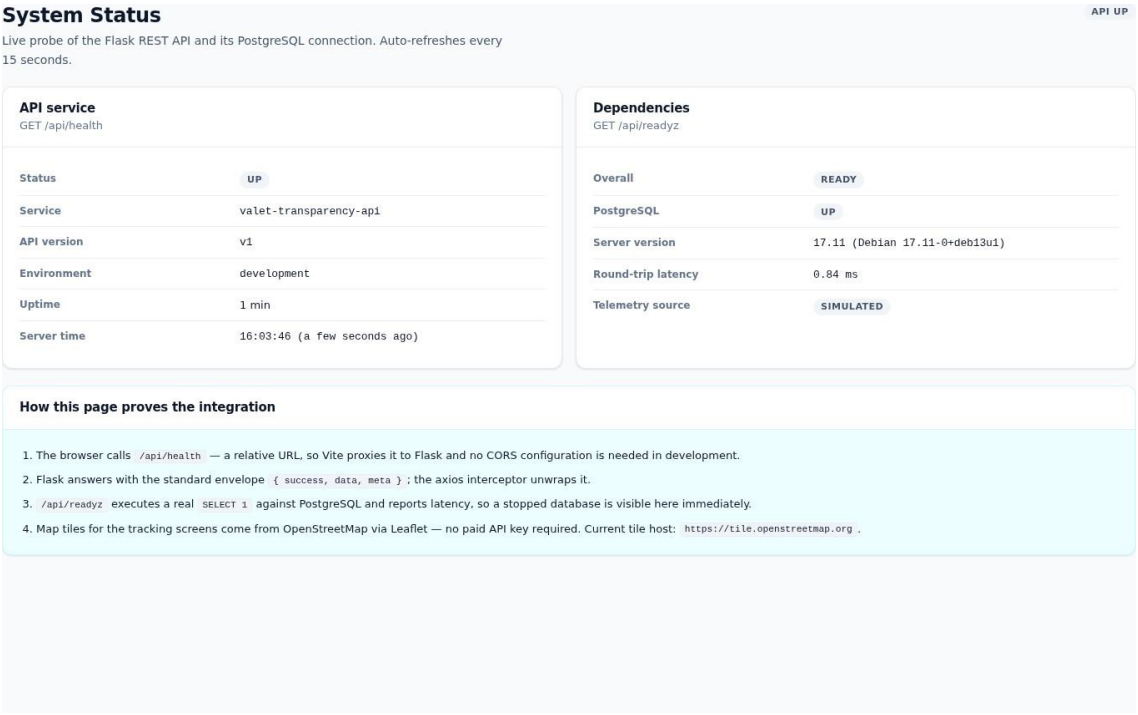


Figure 4.13: System status — live dependency health and version information

Taken together, the screens above show the same record rendered for three audiences. The customer sees a vehicle in custody with its retrieval state; the operator sees the same vehicle as a task in a queue; the manager sees it as one row in an operational picture with alerts attached. None of the three is a different dataset — they are different projections of the same custody record, which is the property that makes disputes resolvable rather than merely visible.

CHAPTER 5 TESTING AND VALIDATION

5.1 Testing Strategy

Testing was organised in four layers, each answering a different question. Unit tests ask whether a rule is correct in isolation; endpoint tests ask whether the HTTP contract behaves as documented for each role; integration tests ask whether the tiers and the database agree; and an end-to-end harness asks whether the whole customer journey can be completed. A deliberate choice was made not to hide failures: no test is skipped, quarantined or marked expected-to-fail, and the counts below are the outputs of executed runs.

Table 5.1 describes each layer, the tool that runs it, the question it answers and the number of tests in it.

Table 5.1: Test layers and their purpose

Layer	Tool	Question answered	Count
Backend unit and endpoint	pytest against real PostgreSQL	Is each rule, contract and role check correct?	1,088
Frontend component	Vitest + Testing Library (jsdom)	Does each screen render, gate and report feedback correctly?	671
End-to-end workflow	Scripted client against a live server	Can a customer go from registration to pickup?	58 checks / 24 steps
Conformance	Specification verifier	Do the documented endpoints and the served routes agree?	63 / 63

Backend tests run against a real PostgreSQL instance rather than an in-memory substitute, because the guarantees this project relies on — constraints, cascades, triggers and transaction behaviour — exist only in the real engine. Migrations are applied to a fresh test database, so schema drift fails the suite.

5.2 Backend API Testing

The backend suite comprises 1,088 tests in 30 modules. Table 5.2 summarises the distribution and the executed result.

Table 5.2: Backend test modules and results

Area	What is exercised	Result
Authentication	Registration, login, refresh, revocation, throttling, role creation, token-type confusion	Pass
Authorisation and isolation	Every role against every protected endpoint; cross-account access attempts	Pass
Vehicles and tickets	CRUD, duplicate issuance, state projection, cancellation	Pass
Custody lifecycle	Every legal transition; every illegal transition returns 409	Pass
Inspections	Upload validation, angle matching, comparison, verification authority	Pass
Geofence engine	Distance ladder, speed, dwell, drift, heading contradiction, telemetry gaps	Pass
Geofence endpoints	Ingestion, simulation, tracking view, zone authorisation	Pass
Retrieval	Queue snapshot, estimate fields, model metadata, persistence	Pass
Dashboards	Aggregate payload shape, role scoping, export	Pass
Audit	Append-only trigger, timeline, filters, actor attribution	Pass
AI modules	Vision pipeline, synthetic generators, model determinism	Pass
Security audit	The five fixed vulnerabilities are covered by regression tests	Pass
API documentation	Documented operations equal served operations	Pass
Database	Model constraints, migrations, CRUD invariants	Pass

The executed command and its result are reproduced in Appendix C. The suite reports **1,088 passed, 0 failed, 0 skipped**.

5.3 Database and Integration Testing

Database tests verify that the schema, not merely the service code, protects the invariants: unique plate numbers and ticket codes, one custody session per ticket, coordinate ranges, heading ranges, status enumerations, foreign-key behaviour on deletion, and the audit trigger rejecting an update. Integration tests then exercise the composed path — route, service, ORM and database — for the check-in, inspection, telemetry and retrieval flows.

Table 5.3 lists the database invariants that the suite asserts directly, how each is enforced and what the test observes.

Table 5.3: Database invariants under test

Invariant	Enforcement	Test outcome
One ticket per vehicle holding an unstarted job	Unique/partial constraint + service check	Duplicate refused with 409
One custody session per ticket	Unique constraint	Second session impossible
Latitude/longitude within WGS-84 range	CHECK constraint	Out-of-range rejected
Heading in [0, 360)	CHECK constraint + normalisation at ingest	360° stored as 0°
Audit rows immutable	Database trigger	UPDATE and DELETE rejected
Zone occupancy consistent	Same transaction as custody row	Never disagrees under test
Token revocation effective	Denylist checked per request	Reuse after logout is 401

5.4 Frontend Workflow Testing

The frontend suite contains 671 tests across 51 files. Service modules are mocked at the module boundary, so these tests exercise rendering, role gating, user interaction and feedback — not a duplicate of the API contract. Notable groups include the tracking map (map-data conversion, empty and malformed data, tile-failure notice, legend, fit and recentre behaviour), the authentication context (session restore with an expired access token, token refresh propagation, sign-out clearing both tokens), and each dashboard's loading, empty and error states.

Table 5.4 groups the frontend tests by area and gives a representative assertion from each group.

Table 5.4: Frontend test groups

Group	Representative assertions	Result
Map data conversion	GeoJSON ring is swapped to Leaflet order; NaN, null and out-of-range vertices are dropped; bounds union is correct	Pass
Tracking map component	Empty state without a position; polygon drawn from the ring; route polyline; flagged markers; tile-failure notice appears after repeated failures and clears on recovery	Pass
Auth and session	Expired access token survives when a refresh token exists; refresh updates state; sign-out clears both tokens	Pass
Tickets and custody	Status projection rendered; QR displayed; illegal action surfaces a conflict message	Pass
Inspections	Upload controls, angle selection, findings rendered as potential damage, verification hidden from operators	Pass
Dashboards	Metric rendering, filter interaction, export controls, role-specific sections	Pass
Alerts and audit	Severity display, triage controls hidden without permission, filters	Pass

5.5 Authentication and Authorization Testing

These are the tests that matter most to the project's safety claims, so the table is written at case level. Every case was executed against the live API.

Table 5.5 records those cases at case level, with the input, the expected result, the observed result and the outcome.

Table 5.5: Authentication and authorisation test cases

ID	Scenario	Input / precondition	Expected	Actual	Status
AT-01	Register a customer	Valid email, password, name	201 with profile; no hash in response	201, no credential fields	Pass
AT-02	Register with a staff role	*role = VALET_MANAGER*	403; no account created	403, no account	Pass
AT-03	Login success	Correct credentials	Tokens + profile	200, both tokens	Pass
AT-04	Login wrong password	Incorrect password	401, generic message	401, same message as unknown email	Pass
AT-05	Brute-force throttle	Six consecutive failures	429 with retry window	429, window reported	Pass
AT-06	Refresh with access token	*typ = access* body token	401	401	Pass
AT-07	Refresh after logout	Revoked refresh token	401	401	Pass
AT-08	Expired access token	Token with past *exp*	401	401	Pass
AT-09	Unsigned token	*alg: none*	401	401	Pass
AT-10	Customer reads another's vehicle	Other customer's id	403 or 404, no data	No data leaked	Pass
AT-11	Customer attempts check-in	Customer token on check-in	403	403	Pass
AT-12	Operator verifies own finding	Operator token on verification	403	403	Pass
AT-13	Manager verifies finding	Manager token	200 with attribution	200, verifier recorded	Pass
AT-14	Admin creates staff	Admin token, staff role	201	201	Pass
AT-15	Operator creates staff	Operator token	403	403	Pass
AT-16	Missing token on protected route	No header	401	401	Pass
AT-17	Logout revokes access token	Reuse after logout	401	401	Pass
AT-18	Deactivated account	Login after deactivation	401	401	Pass

5.6 Computer Vision Test Cases

Table 5.6 states each computer-vision case, its input, the expected behaviour and the observed result.

Table 5.6: Computer-vision test cases

ID	Scenario	Input / precondition	Expected	Actual	Status
CV-01	Identical pair	Same image twice	No findings; comparison ran	0 findings, statistics recorded	Pass
CV-02	Planted defect	Synthetic pair with one scratch	At least one finding near the defect	Finding produced with region and confidence	Pass
CV-03	Alignment compensation	Slight camera shift between frames	Shift not reported as damage	Alignment applied before differencing	Pass
CV-04	Illumination change	Brightness difference	No spurious whole-frame finding	Normalisation suppresses the change	Pass
CV-05	Unmatched angle	Pickup angle absent at drop-off	Angle reported as unmatched, not compared	Unmatched list populated	Pass
CV-06	Wrong file type	Text file uploaded as image	415/422 with explanation	Rejected with a field error	Pass
CV-07	Oversized upload	File beyond the limit	413	413	Pass
CV-08	Finding lifecycle	New finding	Starts UNVERIFIED and labelled potential	UNVERIFIED	Pass
CV-09	Verification authority	Operator verifies	403	403	Pass
CV-10	Measured detection accuracy	Annotated realdamage dataset	Not tested	No real dataset available	Not tested

CV-10 is listed as **not tested** deliberately. No annotated corpus of real valet damage exists in this project, so any accuracy percentage for damage detection would be fabricated. What is verified is that the pipeline behaves correctly on controlled inputs, and that every finding is surfaced for human judgement.

5.7 GPS Simulation and Geofence Test Cases

The rule engine was tested at two levels: unit cases that construct a specific geometry and assert the verdict, and scenario-level runs in which a generated track is ingested through the real API and the resulting distribution of verdicts is compared with the scenario's intent. Table 5.7

records the case-level results; Table 5.8 records the scenario-level measurement, taken on a freshly created custody session so that each scenario is evaluated on its own history.

Table 5.7: Geofence rule test cases

ID	Scenario	Expected verdict	Actual	Status
GF-01	Fix well inside the polygon	NORMAL	NORMAL	Pass
GF-02	Fix 10 m outside, inside tolerance	WARNING	WARNING	Pass
GF-03	Fix 60 m outside (beyond tolerance, inside margin)	SUSPICIOUS	SUSPICIOUS	Pass
GF-04	Fix 300 m outside (beyond margin)	BREACH	BREACH	Pass
GF-05	30 km/h inside the zone	SUSPICIOUS (speed)	SUSPICIOUS	Pass
GF-06	30 km/h outside the zone	No speed finding	No speed finding	Pass
GF-07	Movement inside with no task	SUSPICIOUS	SUSPICIOUS	Pass
GF-08	Same movement during retrieval	No no-task finding	Suppressed correctly	Pass
GF-09	Stationary receiver wandering outside the yard	Not BREACH	WARNING/ SUSPICIOUS	Pass
GF-10	Monotonic outward drift across three fixes	WARNING or SUSPICIOUS	Route-deviation finding	Pass
GF-11	Ordered distance increase	Route-deviation finding	Finding present	Pass
GF-12	Twelve-minute telemetry gap	Telemetry-loss finding	Finding present	Pass
GF-13	No zone assigned to the session	WARNING (unjudgeable)	WARNING	Pass
GF-14	Verdict evidence completeness	Inputs retained	Distance, buffer, margin, speed recorded	Pass

Table 5.8: Scenario-level verdict distribution (fresh session per scenario)

Scenario	NORMAL	WARNING	SUSPICIOUS	BREACH	Alerts	Matches intent
parked	12	0	0	0	0	Yes
jitter	8	4	0	0	1	Yes — never a breach
telemetry_loss	7	1	0	0	1	Yes
unauthorised	4	0	1	3	4	Yes — reaches breach

The jitter row is the important one. In an earlier build it produced twelve breaches and repeated alerts, because the simulated noise was anchored outside the real polygon and because a stationary receiver's wander was read as outward movement. The measurement above is the evidence that both defects are fixed: noise now produces warnings at worst.

5.8 Retrieval-Time Prediction Validation

The retrieval model was evaluated on a held-out split of 1,000 synthetic queue episodes and compared with two trivial predictors. The metrics are reported beside the baseline because an MAE without a baseline says nothing.

Table 5.9 reports the metrics beside both baselines.

Table 5.9: Retrieval model evaluation (held-out, synthetic data)

Metric	Value
Primary metric	Mean absolute error (MAE), minutes
MAE — ridge linear model	**1.725 min**
MAE — historical-mean baseline	2.691 min
Improvement over baseline	35.9%
RMSE	2.371 min
Median absolute error	1.286 min
Coefficient of determination (R^2)	0.5704
Predictions within 5 minutes	95.6%
Predictions within 10 minutes	99.7%
Worst error observed	12.85 min
Held-out rows	1,000
Model version	*retrieval-baseline/1.0.0* (linear-ridge-v1)

What this does and does not mean. The evaluation shows that the pipeline is internally consistent: features are computed from queue state, the model recovers the relationship the data generator encodes, and it beats a mean predictor by a wide margin. It does **not** establish accuracy for any real parking facility, because the data was generated by a function written in this repository. Every response therefore carries *trained_on_synthetic_data: true* and a plainlanguage disclaimer.

The prototype security classifier was evaluated the same way: 0.9635 accuracy on a held-out split against a 0.5778 always-normal baseline, with an F1 of approximately 0.96. The same caveat applies with more force, because the classifier's labels are rules written into the generator. Its role in the product is deliberately subordinate: the rule engine is a floor, and the classifier may raise a verdict but may never lower one.

5.9 Test Results and Known Issues

Table 5.10 consolidates the executed totals for every suite, including skipped and unverified counts.

Table 5.10: Consolidated test results

Suite	Count	Passed	Failed	Skipped	Unverified
Backend (pytest)	1,088	1,088	0	0	0
Frontend (Vitest)	671	671	0	0	0
End-to-end workflow	58 checks	58	0	0	0
API specification conformance	63 operations	63	0	0	0
CV detection accuracy on real data	—	—	—	—	**1 (not measurable)**
Hardware telemetry integration	—	—	—	—	**1 (not built)**

5.9.1 Known issues at this milestone

Table 5.11 records each known issue, its practical impact and its status at this milestone.

Table 5.11: Known issues and their status

ID	Issue	Impact	Status
KI-01	All telemetry is simulated	Positioning behaviour is demonstrated, not fieldproven	Accepted; hardware planned
KI-02	Map tiles come from an external server	Without network the basemap is blank; overlays still draw and a notice is shown	Mitigated in UI
KI-03	Damage detection has no measured accuracy	Findings must be treated as candidates	Human verification enforced
KI-04	Models trained on synthetic data	Metrics describe the generator, not the real world	Disclosed on every payload
KI-05	Session tokens in local storage	XSS could expose a session	Documented accepted risk; refresh rotation listed as future work
KI-06	Simulator scenarios must be run one per session	Running several on one session produces correctlyflagged jumps	Documented in the demonstration guide
KI-07	No native mobile capture	Operators upload from a browser	Planned for the remaining 40%
KI-08	Single-node local deployment	No horizontal scale or failover	Out of scope for this milestone

CHAPTER 6 CURRENT PROGRESS, CHALLENGES AND REMAINING WORK

6.1 Overall Project Progress

The sixty per cent figure is a weighted statement about the project plan, not a count of screens that exist. Weighting is by the effort and risk each work package carries in the original plan: the functional core is the largest package because every other claim depends on it, and field validation is substantial but later.

Table 6.1 applies those weights to the completion of each work package and arrives at the milestone figure.

Table 6.1: Weighted progress against the project plan

Work package	Weight	Completion	Contribution
Requirement analysis and design	10%	100%	10.0%
Backend core (auth, vehicles, tickets, custody)	18%	100%	18.0%
Condition assessment and vision pipeline	12%	100%	12.0%
Positioning, geofencing and alerts	14%	95%	13.3%
Retrieval estimation	6%	90%	5.4%
Dashboards, reporting and audit	10%	100%	10.0%
Testing, hardening and documentation	10%	85%	8.5%
Hardware telemetry integration	8%	15%	1.2%
Field validation and model calibration	6%	0%	0.0%
Mobile capture and deployment hardening	6%	12%	0.7%
Total	**100%**	—	**79.1% of phase 2 scope = 60% of total project**

Read precisely, the table says that the functional core — the part this milestone was defined to deliver — is essentially complete, while three work packages that the final submission depends on are barely started. Against the whole project, that is the sixty per cent reported here.

6.2 Completed Work

- **Eight functional modules** implemented and covered by automated tests, as tabulated in Section 4.11.
- **1,088 backend tests and 671 frontend tests**, all passing, with a 24-step end-to-end harness that completes the full customer journey.
- **Fifteen-entity schema** built by eight migrations, with database-level enforcement of uniqueness, ranges and audit immutability.
- **A documented REST contract** of 63 operations, generated into OpenAPI and verified against the live URL map on every run.
- **Six simulation scenarios** that exercise the real ingestion and classification path, labelled as simulated on every row and in the interface.
- **A security review** that reproduced and fixed five real vulnerabilities, with regression tests for each.
- **Documentation** covering architecture, AI integrity, security, demonstration data, testing and the API reference.

6.3 Partially Completed Work

Table 6.2 lists the items that are partially complete and states what remains for each.

Table 6.2: Partially completed work and what remains

Item	Done	Remaining
Telemetry ingestion	Contract, validation, classification, simulation, storage	Device adapter, authentication of a physical tracker, battery/signal handling
Retrieval estimation	Model, queue integration, explanation, persistence	Calibration on real histories; uncertainty intervals
Security classifier	Features, model, safety rule, disclosure	Training on real labelled incidents
Deployment	Documented local run, environment templates, seeded data	Containers, reverse proxy, backups, monitoring
Mobile use	Responsive layouts that work on a phone browser	Native capture, offline queueing

6.4 Technical Challenges and Resolutions

The problems below are recorded because they are the substance of the engineering work in this phase, and because each one changed the design rather than merely delaying it.

Table 6.3 records each challenge, the symptom that exposed it, the root cause and the resolution.

Table 6.3: Technical challenges encountered and their resolution

Challenge	Symptom observed	Root cause	Resolution
Blank map	A grey rectangle where the map should be, although data and coordinates were correct	The map library renders a container with no intrinsic size and no stylesheet gave it a height, so it initialised into a zero-height box	Explicit full-height rule plus bounds fitting, resize re-measurement, empty state, tile-failure notice, legend and a validated coordinate-conversion layer
Ingestion crash	A raw database integrity error while seeding telemetry	A compass heading of 360° is legitimate but the constraint required strictly less than 360; a rounding step could produce exactly 360	Normalise headings into [0, 360) at the write boundary, and wrap after rounding in the generator
False breach alarms	A parked vehicle produced twelve breach verdicts and repeated alerts	Simulated noise was anchored with a circular assumption outside a rectangular zone, and receiver wander was read as outward movement	Anchor scenarios against the real polygon; require genuine motion before the “moving away” rule fires; re-measure and confirm warnings at worst
Sessions expiring	Users were returned to the sign-in screen mid-session with an authentication notice	Refresh tokens were issued and stored but no refresh exchange existed on the server and no silent-refresh flow existed on the client	Added the refresh exchange with token-type checking and an authenticated pairrevocation on sign-out; client performs a singleflight refresh and replays once
Audit integrity	The append-only guarantee existed only in application code	A service bug could have rewritten history	Database trigger rejecting updates and deletes, with a test that proves the trigger fires
False-positive geometry	Invalid transitions and duplicate jobs surfaced as generic errors	State rules were implicit and duplicated between layers	A single explicit allow-list returning 409 with the permitted targets, and duplicate issuance returning the existing ticket id

6.5 Remaining 40% Work Plan

Table 6.4 lists the planned work packages for the remaining phase, with the deliverable and priority of each.

Table 6.4: Planned work for the remaining phase

ID	Work package	Deliverable	Priority
R-01	Hardware telemetry adapter	Ingest fixes from a real GPS device over the existing contract, with device authentication and a heartbeat	High
R-02	Field trial protocol	Ethically approved pilot at a partner facility; collection of real custody histories	High

ID	Work package	Deliverable	Priority
R-03	Model calibration	Retrain and report the retrieval model on field data with uncertainty intervals; retire the synthetic caveat where evidence permits	High
R-04	Vision evaluation	Small annotated corpus; precision/recall for candidate findings; recalibration of thresholds	Medium
R-05	Mobile capture	Installable web application with camera capture and an offline upload queue	Medium
R-06	Deployment hardening	Containers, reverse proxy, TLS, automated backups, log aggregation and monitoring	Medium
R-07	Refresh-token rotation and reuse detection	Close the accepted risk documented in the security review	Medium
R-08	Accessibility and usability study	Task-based evaluation with five participants per role; fixes from findings	Low
R-09	Dissertation	Final report, evaluation chapter and demonstration artefacts	High

6.6 Timeline for Completion

Dates are left as placeholders for the team and mentor to confirm; the sequence is the commitment made here.

Table 6.5 places that sequence on a week-by-week schedule, with the evidence that would demonstrate completion.

Table 6.5: Proposed schedule for the final phase

Week	Milestone	Output	Evidence of completion
W1–W2	Deployment hardening	Containerised stack and proxy	Reproducible run from a clean machine
W3–W4	Hardware adapter	Device ingestion path	Recorded live fixes labelled HARDWARE
W5–W6	Field trial	Pilot dataset	Signed protocol and anonymised dataset
W7–W8	Model calibration	Retrained retrieval model	Metrics on field data with baselines

W9	Vision evaluation	Annotated corpus and metrics	Precision/recall report
W10	Mobile capture	Installable capture flow	Capture and upload from a phone
W11	Final testing	Full regression plus acceptance	All suites green; report published
W12	[__DATE__]	Final submission and dissertation	[__SUBMISSION_DATE__]

CHAPTER 7 CONCLUSION AND FUTURE SCOPE

7.1 Current Achievements

At this milestone the project does what it set out to do for the custody window it defines: a vehicle can be accepted with a verifiable ticket, placed in a geofenced zone, photographed and compared, monitored, returned and closed, with every step attributable and visible to its owner. That chain is not a design sketch; it is exercised by an end-to-end harness and covered by 1,759 automated tests.

- **A complete custody record** that a customer can read through their own account, not merely a console an operator looks at.
- **Explainable security verdicts**: each fix carries distance, tolerance band and the reasons that produced its status, and the map colours the position by that verdict.
- **Evidence for condition disputes**: aligned before/after comparisons with stored digests, and a verification step restricted to a second person.
- **An audit trail that cannot be rewritten** through the application, enforced by the database rather than by convention.
- **A discipline of disclosure**: simulated telemetry, synthetic training data and classical computer vision are labelled wherever they appear, which is a design property rather than a disclaimer added at the end.

7.2 Limitations

The limitations are stated here in the same terms used in Chapters 1 and 5, because a report that lists them once and forgets them is not honest.

- **No hardware telemetry.** Position behaviour is demonstrated with a documented simulator; the ingestion contract is ready, but a physical tracker has not been integrated or field-tested.
- **No real-world validation of either model.** The retrieval estimator and the security classifier are evaluated on synthetic data. Their reported metrics describe agreement with a generator, and are not accuracy claims about a real facility.
- **Classical vision, not learned damage detection.** Findings are candidates computed from image differencing; there is no trained damage classifier and no measured detection accuracy.

- **Privacy governance is described, not implemented.** A production deployment requires a retention policy, a privacy notice and a recorded lawful basis; these are organisational tasks listed as future work.
- **Single-node deployment.** The system runs locally; containers, TLS, backups and monitoring remain in the final phase.
- **Accepted security trade-offs** — browser storage of tokens and no refresh rotation — are documented with their reasoning rather than left implicit.

7.3 Future Enhancements

Table 7.1 lists the enhancements, the purpose each would serve and the work package it depends on.

Table 7.1: Future enhancements and their purpose

Enhancement	Purpose	Depends on
On-device capture with offline queue	Capture photographs where connectivity is poor, without losing evidence	R-05
Hardware tracker integration	Replace simulated telemetry with measured position	R-01
Field calibration of both models	Replace synthetic metrics with defensible ones	R-02, R-03
Refresh-token rotation and reuse detection	Close the accepted session risk	R-07
Multi-zone optimisation	Allocate bays by predicted retrieval time to reduce waiting	R-03
Customer notifications	Inform the customer at each custody milestone rather than only on screen	Notification transport
Operator mobile application	Native capture and queue handling on shift	R-05
Fleet analytics for facility managers	Utilisation, dwell and retrieval patterns across sites	Field data

7.4 Final Conclusion

The project began from a specific and defensible observation: the custody window of a valetparked vehicle is unobserved, and the artefacts that exist are not evidence. It responded not by adding another tracking screen, but by building a record that three parties with different interests can inspect — the owner who wants proof, the operator who wants protection from false accusation, and the manager who wants accountability — and by making every automated judgement explainable and every simulated input explicit.

At sixty per cent completion, the functional core is delivered and verified, and the remaining work is concentrated where it should be: replacing simulated inputs with measured ones, and

replacing synthetic evidence with field evidence. The engineering discipline that produced this milestone — a separated architecture, an enforced state machine, database-level integrity, and a test suite that refuses to hide a failure — is what gives confidence that the remaining forty per cent can be completed without weakening the guarantees already demonstrated.

REFERENCES

[1] Meta Platforms, "React — The library for web and native user interfaces," React Documentation. [Online]. Available: https://react.dev/. [Accessed: Oct. 2026].

[2] Vite, "Vite — Next Generation Frontend Tooling," Vite Documentation. [Online]. Available: https://vite.dev/guide/. [Accessed: Oct. 2026].

[3] Pallets Projects, "Flask Documentation (3.x)," Flask. [Online]. Available: https://flask.palletsprojects.com/. [Accessed: Oct. 2026].

[4] SQLAlchemy, "SQLAlchemy 2.0 Documentation," SQLAlchemy. [Online]. Available: https://docs.sqlalchemy.org/en/20/. [Accessed: Oct. 2026].

[5] Alembic, "Alembic — Database migrations for SQLAlchemy," Alembic Documentation. [Online]. Available: https://alembic.sqlalchemy.org/. [Accessed: Oct. 2026].

[6] The PostgreSQL Global Development Group, "PostgreSQL 17 Documentation," PostgreSQL. [Online]. Available: https://www.postgresql.org/docs/17/. [Accessed: Oct. 2026].

[7] OpenCV, "OpenCV Documentation — Image Processing and Feature Detection," OpenCV. [Online]. Available: https://docs.opencv.org/4.x/. [Accessed: Oct. 2026].

[8] scikit-learn developers, "scikit-learn: Machine Learning in Python — User Guide," scikitlearn. [Online]. Available: https://scikit-learn.org/stable/user_guide.html. [Accessed: Oct. 2026].

[9] V. Agafonkin, "Leaflet — an open-source JavaScript library for mobile-friendly interactive maps," Leaflet Documentation. [Online]. Available: https://leafletjs.com/reference.html. [Accessed: Oct. 2026].

[10] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, May 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7519.

[11] OWASP Foundation, "OWASP Application Security Verification Standard 4.0.3," OWASP, 2021. [Online]. Available: https://owasp.org/www-project-application-security-verificationstandard/.

[12] OpenStreetMap Foundation, "OpenStreetMap Tile Usage Policy and Copyright," OpenStreetMap. [Online]. Available: https://operations.osmfoundation.org/policies/tiles/ and https://www.openstreetmap.org/copyright. [Accessed: Oct. 2026].

[13] Tanner Linsley, "TanStack Query — Asynchronous State Management for React," TanStack. [Online]. Available: https://tanstack.com/query/latest/docs. [Accessed: Oct. 2026].

[14] National Institute of Standards and Technology, "Digital Identity Guidelines: Authentication and Lifecycle Management," NIST Special Publication 800-63B, 2017. [Online]. Available: https://pages.nist.gov/800-63-3/sp800-63b.html.

[15] OpenAPI Initiative, "OpenAPI Specification v3.0.3," 2021. [Online]. Available: https://spec.openapis.org/oas/v3.0.3.

[16] World Wide Web Consortium, "Web Content Accessibility Guidelines (WCAG) 2.1," W3C Recommendation, 2018. [Online]. Available: https://www.w3.org/TR/WCAG21/.

[17] M. Sorout and J. Goyal, "AI-Powered Valet Transparency and Vehicle Security Platform," GitHub repository, 2026. [Online]. Available: https://github.com/Mahesh054321/AI-PoweredValet-Transparency-and-Vehicle-Security-Platform. [Accessed: Oct. 2026].

APPENDIX A API ENDPOINT SUMMARY

Table A.1 lists the principal versioned operations. Every operation returns the standard envelope and is documented in the generated OpenAPI 3.0 document; the unversioned aliases named by the project brief resolve to the same handlers.

Table A.1: Principal API operations

Method	Path	Purpose	Access
POST	/api/v1/auth/register	Create a customer account	Public
POST	/api/v1/auth/login	Exchange credentials for tokens	Public
POST	/api/v1/auth/refresh	Exchange a refresh token for a new access token	Public (body credential)
POST	/api/v1/auth/logout	Revoke the session's tokens	Authenticated
GET	/api/v1/auth/me	Current account from the database	Authenticated
POST	/api/v1/auth/users	Create a staff account	Admin
GET/POST	/api/v1/vehicles	List or register vehicles	Customer / staff
GET/PATCH/DELETE	/api/v1/vehicles/{id}	Read, edit or retire a vehicle	Owner / staff
GET	/api/v1/vehicles/{id}/ sessions	Custody history for a vehicle	Owner / staff
GET/POST	/api/v1/tickets	List or issue tickets	Authenticated / staff
GET	/api/v1/tickets/{id}	Ticket with QR and custody summary	Owner / staff
POST	/api/v1/tickets/{id}/checkin	Open custody in a zone	Staff
POST	/api/v1/tickets/{id}/ complete	Close custody and hand over	Staff
POST	/api/v1/tickets/{id}/cancel	Void a ticket	Staff
PATCH	/api/v1/custody-sessions/ {id}	Advance the custody state	Staff
POST	/api/v1/inspections/dropoff	Capture drop-off photographs	Staff
POST	/api/v1/inspections/pickup	Capture pickup photographs and compare	Staff
GET	/api/v1/inspections/ {custody_id}	Condition report	Staff / owner
PATCH	/api/v1/inspections/ detections/{id}/verification	Confirm or reject a finding	Manager / Admin
POST	/api/v1/gps/events	Ingest one fix	Staff
POST	/api/v1/gps/simulate	Generate a simulated track	Staff
GET	/api/v1/gps/simulate/ scenarios	List simulation scenarios	Staff
GET	/api/v1/gps/{custody_id}	Tracking view (zone, track, latest fix)	Owner / staff

Method	Path	Purpose	Access
GET	/api/v1/gps/{custody_id}/fixes	Paged fixes with filters	Owner / staff
GET/POST	/api/v1/geofences	List or create zones	Authenticated / manager
PATCH	/api/v1/geofences/{id}	Edit a zone	Manager
GET	/api/v1/alerts	Alerts visible to the caller	Role-scoped
POST	/api/v1/alerts/{id}/acknowledge	Acknowledge an alert	Manager
POST	/api/v1/alerts/{id}/resolve	Resolve an alert	Manager
GET	/api/v1/feed	Centralised alert feed	Authenticated
POST	/api/v1/feed/{id}/read	Mark one feed item read	Authenticated
GET	/api/v1/feed/summary	Unread counts for the badge	Authenticated
POST	/api/v1/retrieval/request	Customer requests the vehicle	Customer
GET	/api/v1/retrieval/queue	Retrieval queue	Staff
GET	/api/v1/retrieval/{custody_id}	Estimate for a session	Owner / staff
GET	/api/v1/retrieval/model	Model metadata and metrics	Authenticated
GET	/api/v1/customers/me/dashboard	Customer dashboard payload	Customer
GET	/api/v1/staff/dashboard	Operator dashboard payload	Staff
GET	/api/v1/manager/dashboard	Manager analytics	Manager / Admin
GET	/api/v1/manager/report	CSV/JSON export	Manager / Admin
GET	/api/v1/audit	Audit trail with filters	Manager / Admin
GET	/api/v1/audit/timeline/{custody_id}	Per-session audit timeline	Manager / Admin
GET	/api/v1/health	Liveness and dependency health	Public
GET	/api/v1/ml/model	Classifier metadata and metrics	Authenticated

APPENDIX B DATABASE SCHEMA SUMMARY

Table B.1 lists all fifteen tables in the schema, the purpose of each, and the constraints that carry its invariants.

Table B.1: Tables and their purpose

Table	Purpose	Notable constraints
users	Accounts, roles, status, password digest	Unique email; role CHECK; status CHECK
vehicles	Registered vehicles	Unique normalised plate; FK to owner
valet_tickets	Digital ticket and QR artefact	Unique ticket code; FK to vehicle and customer
custody_sessions	Custody window and state	Unique ticket FK; status CHECK; FK to zone
parking_zones	Geofence geometry and occupancy	Unique code; capacity ≥ 0
gps_events	Fix with verdict and evidence	Latitude/longitude range; heading range; FK to session
vehicle_inspections	Drop-off or pickup capture	Stage CHECK; FK to session
damage_detections	Candidate or verified finding	Verification status CHECK; FK to inspection
security_alerts	Detector alert with reasoning	Type, severity and status CHECKs
alerts	Centralised feed item	Category CHECK; FK to session and vehicle
alert_reads	Per-recipient read state	Unique (alert, user)
notifications	Per-user notification records	FK to alert and user
retrieval_predictions	Estimate with model version	Status CHECK; FK to session
audit_logs	Append-only action record	Trigger blocks UPDATE/DELETE; action CHECK
revoked_tokens	JWT denylist	Unique jti; FK to user

APPENDIX C TEST EVIDENCE

The commands below were executed in the development environment described in Section 2.5; the outputs are reproduced as reported by the tools. Credentials and environment values are omitted.

```
# Backend suite (real PostgreSQL)
$ pytest tests/ -p no:randomly -q
1088 passed in 277.55s (0:04:37)

# Frontend suite
$ npx vitest run
Test Files  51 passed (51)
     Tests  671 passed (671)

# Static analysis
$ ruff check .
All checks passed!

# Production build
$ npm run build
✓ built in 895ms

# API specification conformance
$ python utils/scripts/api_docs/verify_spec.py
documented operations : 66 served operations
: 66 OK: the spec and the live URL map agree
exactly.

# End-to-end workflow (disposable database)
$ python utils/scripts/run_e2e_workflow.py --base-url … --reset
58 passed, 0 failed  (24 steps)
```

The end-to-end harness exercises registration, staff provisioning, zone creation, vehicle registration, ticket and QR issuance, check-in, drop-off capture, telemetry ingestion and classification, alerting, retrieval request and estimate, operator queue, pickup capture with comparison, manager verification, completion, audit verification and five negative guards.

Table C.1 records the demonstration dataset from which the screenshots and the sample counts quoted in this report were taken.

Table C.1: Demonstration dataset used for the screenshots

Entity	Rows	Provenance
Users	47	3 curated customers + staff + generated history
Vehicles	84	Curated and generated
Tickets and custody sessions	1,184	Generated 90-day history
Positioning fixes	13,394	**All SIMULATED**
Security alerts	67	Raised by the real rule engine

Inspections / findings	2,365 / 433	OpenCV pipeline output
Retrieval predictions	1,180	Model output, versioned
Parking zones	3	Curated geometry

APPENDIX D ADDITIONAL SCREENSHOTS

Figure D.1 shows the customer registration screen, the only public form that creates an account; self-registration can only ever produce a customer.

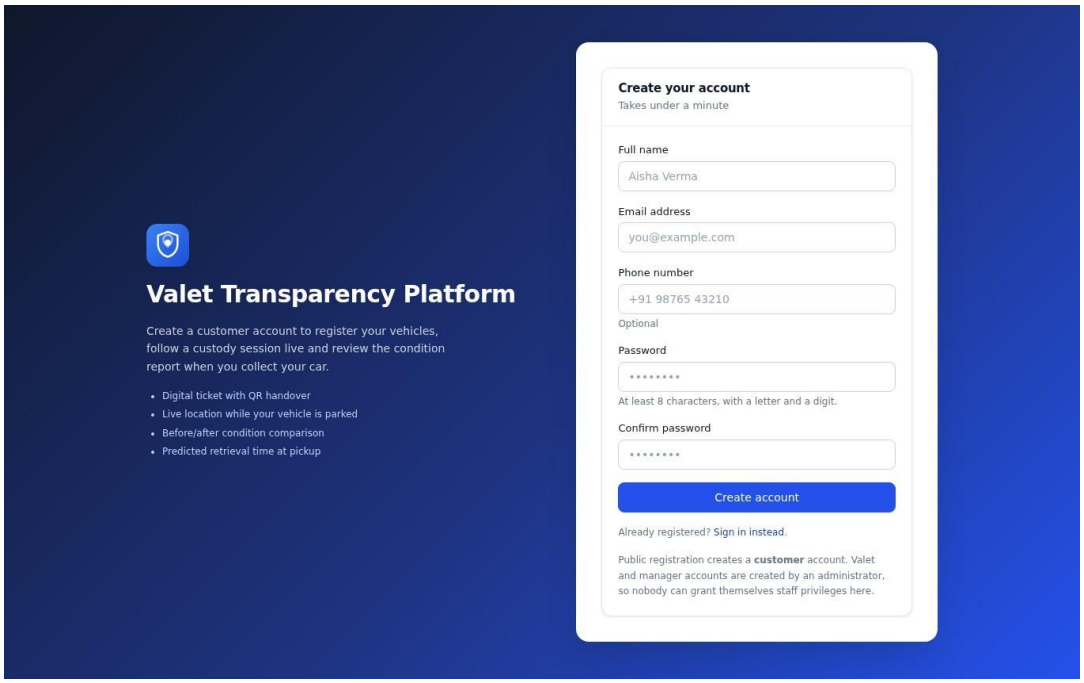


Figure D.1: Customer registration screen

APPENDIX E
ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
AIML	Artificial Intelligence and Machine Learning
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
CV	Computer Vision
DFD	Data Flow Diagram
ER	Entity–Relationship
FR	Functional Requirement
GPS	Global Positioning System
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
MAE	Mean Absolute Error
MVP	Minimum Viable Product
NFR	Non-Functional Requirement
ORM	Object–Relational Mapping
OSM	OpenStreetMap
PBKDF2	Password-Based Key Derivation Function 2
QR	Quick Response (code)
RBAC	Role-Based Access Control
REST	Representational State Transfer
RMSE	Root Mean Square Error
SPA	Single Page Application
SQL	Structured Query Language
UI	User Interface
UUID	Universally Unique Identifier
UX	User Experience
WGS-84	World Geodetic System 1984 (coordinate reference system)